

ALAZAR PRD MASTER ENTERPRISE v11

Fintech production-grade product requirements, system architecture, auditability, APIs, ledger, delivery, settlement, observability and resilience

Documento Word maestro para CTO, desarrollo MVP, inversión, operación y auditoría técnica

Versión v11 construida acumulativamente sobre el PRD maestro v9 y los módulos Purchase/Draw desarrollados previamente. La regla de esta versión es expandir y profundizar sin destruir contenido útil: toda función se conecta con roles, procesos, datos, auditoría, APIs y eventos.

Índice ejecutivo del documento

PARTE I - Negocio y visión

PARTE II - Arquitectura del sistema

PARTE III - Purchase Engine

PARTE IV - Draw Engine

PARTE V - Delivery + Settlement

PARTE VI - Auditoría y compliance

PARTE VII - APIs enterprise

PARTE VIII - Infraestructura y observabilidad

PARTE IX - QA, resiliencia y roadmap

PARTE I - NEGOCIO Y VISIÓN

1. Visión estratégica y razón del negocio

ALAZAR es un marketplace de sorteos digitales que permite a Clientes monetizar bienes, servicios o experiencias mediante tickets, mientras los Usuarios adquieren participación probabilística bajo una experiencia simple y de bajo costo de entrada. La plataforma no debe interpretarse como una app informal de rifas, sino como un sistema transaccional con lógica fintech-ready: cada pago debe reconciliarse, cada ticket debe ser trazable, cada sorteo debe ser reproducible y cada decisión operativa debe quedar auditada.

La razón de negocio es convertir activos difíciles de vender de forma directa en campañas masivas de participación. Esto crea valor para el Cliente porque amplía alcance, para el Usuario porque reduce barrera de acceso a premios de alto valor, y para ALAZAR porque captura un fee de plataforma sobre una operación estructurada. La confianza es el activo principal: sin auditoría, fairness y control financiero, el modelo no escala.

C4 Context - ALAZAR Enterprise Platform

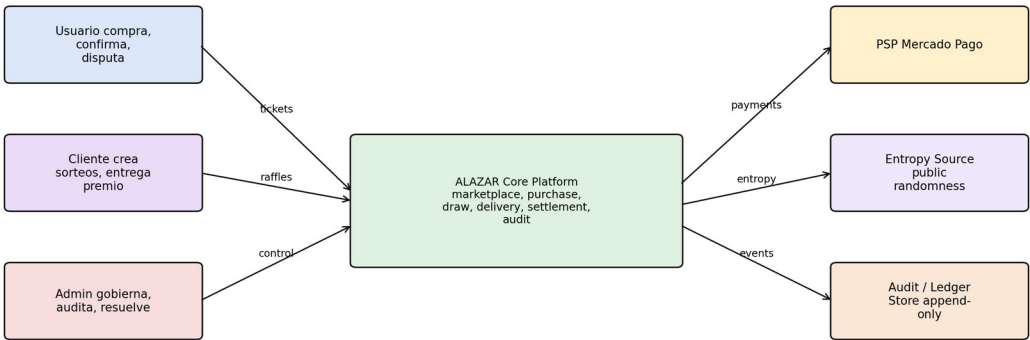


Figura 1. C4 Context general de ALAZAR como plataforma marketplace fintech-ready.

2. Trust model y principios de diseño

El trust model de ALAZAR se basa en cuatro garantías: integridad financiera, integridad probabilística, control administrativo auditado y trazabilidad forense. La integridad financiera exige que no exista ticket sin pago consolidado o sin registro ledger; la integridad probabilística exige que el ganador pueda reproducirse con pool_hash, entropía externa y algoritmo versionado; el control administrativo exige que cualquier override requiera motivo y registro; y la trazabilidad forense exige que el trace_id conecte APIs, eventos, DB, logs, ledger y auditoría.

Principio	Aplicación práctica	Evidencia esperada
Auditabilidad por diseño	Cada acción sensible genera audit_event append-only	trace_id, actor, entity_id, payload_hash
Consistencia financiera	Cada movimiento se registra en doble entrada	ledger_entries balanceadas
Fairness verificable	Draw usa pool canónico y entropía externa	draw_proof reproducible
Control administrativo	Admin puede intervenir, pero con reason_required	ADMIN_OVERRIDE_RECORDED
Escalabilidad modular	Dominios separados por bounded context	eventos y APIs por módulo

3. Roles, RBAC y funciones por tipo de usuario

El sistema se organiza en tres perfiles primarios: Usuario, Cliente y Admin. Esta separación es de autoridad, no solo de interfaz. El Usuario consume sorteos, compra tickets, consulta resultados y confirma o disputa entrega. El Cliente crea la

oferta y gestiona la entrega, pero no controla soberanamente el resultado. El Admin gobierna aprobación, ejecución excepcional, disputa, auditoría y settlement.

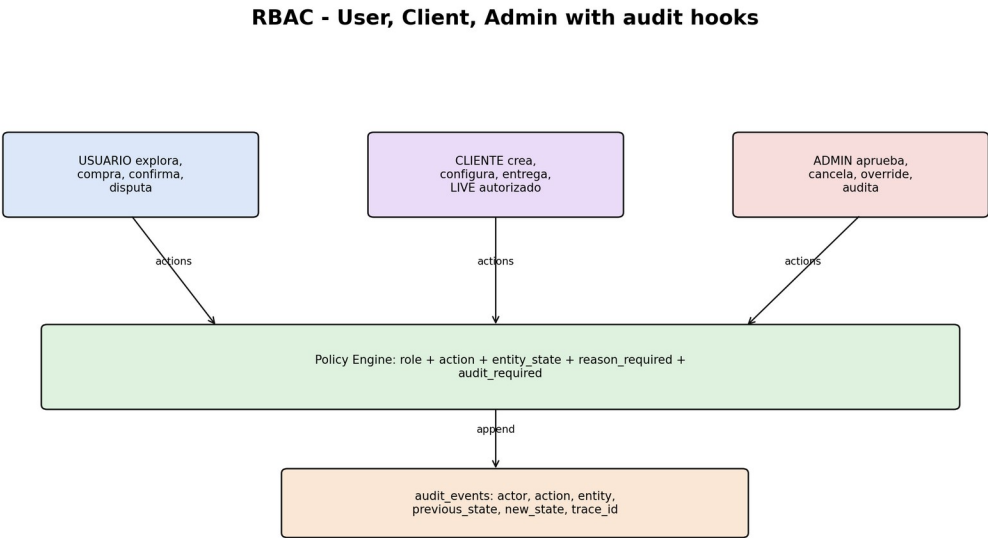


Figura 2. Modelo RBAC sensible a estado con auditoría obligatoria.

Proceso / Función	Usuario	Cliente	Admin	Auditoría obligatoria
Autenticación	Registro, login, recuperar acceso	Onboarding cliente	Gestión de usuarios internos	AUTH_LOGIN, USER_CREATED
Sorteos	Ver sorteos, detalle, filtros	Crear draft, editar, enviar revisión	Aprobar, observar, cancelar	RAFFLE_CREATED, RAFFLE_APPROVED
Compra	Comprar, pagar, ver tickets	Consultar ventas propias	Soporte y auditoría	ORDER_INTENT_CREATED, PURCHASE_COMPLETED
Draw	Consultar resultado y proof	Ejecutar LIVE autorizado	Ejecutar, forzar, bloquear	DRAW_REQUESTED, DRAW_EXECUTED
Entrega	Confirmar tenencia o disputar	Subir evidencia	Resolver disputa	DELIVERY_CONFIRMED, DISPUTE_RESOLVED
Settlement	Consultar estado	Consultar neto y payout	Liberar, congelar, revertir	SETTLEMENT_EXECUTED

3.1 Usuario

El Usuario debe poder descubrir sorteos, entender precio del ticket, comprar sin fricción, recibir tickets verificables, consultar resultados, validar la prueba del sorteo, confirmar entrega o abrir disputa. Cada acción se enlaza con user_id, order_id, raffle_id y trace_id. Esta granularidad permite reconstruir la experiencia completa y defender cualquier decisión ante soporte o auditoría.

3.2 Cliente

El Cliente administra la oferta: crea sorteos, define activo, usa el pricing engine, selecciona tipo de sorteo, envía a revisión, monitorea ventas y entrega el premio. Su autonomía está limitada por aprobación Admin y por reglas de ejecución auditada, especialmente en modo LIVE. Esto protege a la plataforma y a los usuarios frente a manipulación o mala operación.

3.3 Admin

El Admin es el rol soberano. Puede aprobar, observar, cancelar, ejecutar Draw, resolver disputa y forzar Settlement. Sin embargo, su poder no es opaco: cada intervención sensible exige reason_required, actor_id, previous_state, new_state y audit_event. La trazabilidad de Admin es indispensable porque sus acciones impactan dinero, resultados y reputación.

```
def authorize(actor, action, entity):
    policy = get_policy(actor.role, action, entity.state)
    if actor.role == 'ADMIN' and policy.requires_reason:
        require_reason(actor, action, entity)
    if not policy.allowed:
        raise AuthorizationError(action)
    append_audit_event('AUTHZ_DECISION', entity.type, entity.id, actor, entity.state, entity.state,
{'action': action, 'allowed': True})
    return True
```

4. Modelo económico, pricing y prospect economy

El modelo económico de ALAZAR equilibra target del Cliente, fee ALAZAR, costo PSP y conversión de Usuario. El MVP mantiene fee ALAZAR fijo de 20%, PSP estimado de 3.99%, redondeo a múltiplos de 10 y tickets discretos de 1, 5, 10, 20 y 50 soles. La capa conductual busca reducir fricción de compra y mejorar percepción de oportunidad, sin engañar ni ocultar información relevante.

Parámetro	Regla MVP	Evolución futura
Fee ALAZAR	20% fijo	por categoría, cliente o madurez
Fee PSP	3.99% referencial	parametrizable por país/proveedor
Redondeo	múltiplo de 10	por moneda/mercado
Tickets	1, 5, 10, 20, 50	sets por categoría y elasticidad
Split	post-confirmación de entrega	escrow/split real automatizado

```
gross = target_cliente / (1 - fee_alazar - fee_psp)
gross_rounded = ceil(gross / 10) * 10
for ticket_price in [1, 5, 10, 20, 50]:
    total_tickets = gross_rounded / ticket_price
    score = conversion_score(ticket_price, total_tickets, prize_value)
    classify_scenario(ticket_price, total_tickets, score) # viral, balance, premium
```

Target cliente	Gross calculado	Gross redondeado	Ticket sugerido	Tickets estimados	Lectura UX
S/ 1,000	S/ 1,315.62	S/ 1,320	S/ 5	264	balance alto atractivo
S/ 4,500	S/ 5,920.27	S/ 5,930	S/ 5 o S/ 10	1,186 / 593	viral o balance
S/ 20,000	S/ 26,312.33	S/ 26,320	S/ 10 o S/ 20	2,632 / 1,316	control de volumen
S/ 50,000	S/ 65,780.82	S/ 65,790	S/ 20 o S/ 50	3,289 / 1,316	premium escalable

5. Tipos de sorteo

Los tipos de sorteo se modelan como configuraciones ejecutables. Cada tipo define condición de activación, condición de ejecución, permisos, número de ganadores, riesgo principal y control auditado. El Draw Engine consume draw_config y no debe implementar comportamiento implícito fuera de configuración versionada.

Tipo	Condición	Modo	Ganadores	Control auditado
T1 Estándar	100% tickets vendidos	AUTO/Admin	1	POOL_FROZEN, DRAW_EXECUTED
T2 Threshold	tickets >= mínimo	AUTO/Admin/Cliente autorizado	1	THRESHOLD_APPROVED
T3 Tiempo fijo	fecha/hora final	AUTO/Admin	1	TIME_TRIGGER_AUDITED
T4 Híbrido	tiempo o threshold	AUTO/Admin	1	FIRST_TRIGGER_RECORDED
T5 Flash	ventana corta	AUTO/Admin	1	FLASH_WINDOW_LOGGED
T6 Multi-ganador	condición definida	AUTO/Admin	N	WINNER_RANK_LOCK
T7 Progresivo	milestone alcanzado	AUTO/Admin	N/fases	MILESTONE_AUDIT
T8 LIVE	READY + autorización	Cliente autorizado/Admin	1 o N	LIVE_AUTHORIZED + PROOF

En T8 LIVE, el Cliente puede ejecutar únicamente con autorización Admin. LIVE cambia la experiencia y el actor que gatilla, pero no modifica el algoritmo. El pool_hash, la entropía externa, el seed_material y el proof reproducible son idénticos al modo AUTO.

PARTE II - ARQUITECTURA DEL SISTEMA

6. Arquitectura funcional y modular

ALAZAR se diseña como plataforma modular event-driven. Para MVP puede implementarse como monolito modular con boundaries claros, pero la separación lógica debe respetar dominios: Raffle, Pricing, Purchase, Draw, Delivery, Settlement, Risk, Audit, Notification, PSP Adapter y Admin Backoffice. Esta decisión reduce complejidad de despliegue inicial sin sacrificar escalabilidad futura.

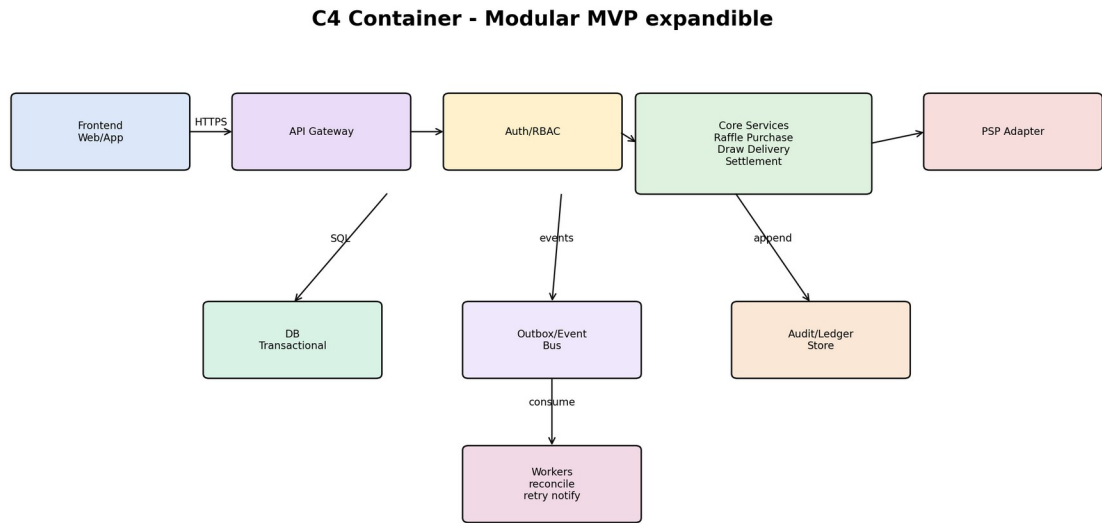


Figura 3. C4 Container del MVP expandible.

Módulo	Responsabilidad	Eventos emitidos	Riesgo controlado
Raffle	Crear, revisar, aprobar, publicar sorteos	RAFFLE_CREATED, RAFFLE_APPROVED	ofertas inválidas
Purchase	Orden, pago, tickets, ledger inicial	ORDER_INTENT_CREATED, PURCHASE_COMPLETED	tickets sin pago
Draw	Freeze pool, proof, winner, certificate	POOL_FROZEN, DRAW_EXECUTED	manipulación de resultado
Delivery	Evidencia, confirmación, disputa	DELIVERY_OPENED, DELIVERY_CONFIRMED	premio no entregado
Settlement	Split, payout, refund, freeze	SETTLEMENT_TRIGGERED, SETTLEMENT_EXECUTED	pago indebido
Audit	Event store y replay	AUDIT_APPEND_OK	falta de trazabilidad

7. Event-driven architecture y outbox

El patrón event-driven permite desacoplar core transaccional de consumidores secundarios como notificaciones, risk, reconciliación y dashboards. La publicación no debe ocurrir directamente en memoria después del commit, sino mediante Outbox Pattern: dentro de la misma transacción se escribe el cambio de dominio, el audit_event y el outbox_event. Un worker publica de forma confiable y reintenta con DLQ si falla.

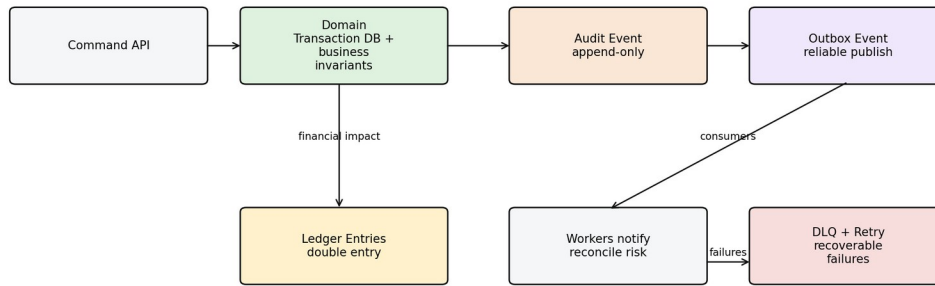
Event-driven architecture with audit-first pattern

Figura 4. Patrón event-driven con audit-first y outbox confiable.

```

def commit_domain_change(command):
    begin_transaction()
    result = apply_business_invariants(command)
    append_audit_event(result.audit)
    insert_outbox_event(result.domain_event)
    if result.financial_impact:
        create_ledger_entries(result.ledger_entries)
    commit_transaction()

# Outbox worker
for event in pending_outbox_events():
    try:
        publish(event)
        mark_published(event.id)
    except RetryableError:
        increment_retry(event.id)
    except FatalError:
        move_to_dlq(event.id)
  
```

8. BPMN y journey end-to-end

El journey completo inicia con creación de sorteo, revisión Admin, publicación, compra, emisión de tickets, ejecución del draw, entrega, eventual disputa y settlement. Cada etapa tiene salida de dominio y evento de auditoría. La auditoría no es un post-proceso; se genera junto con cada cambio de estado.

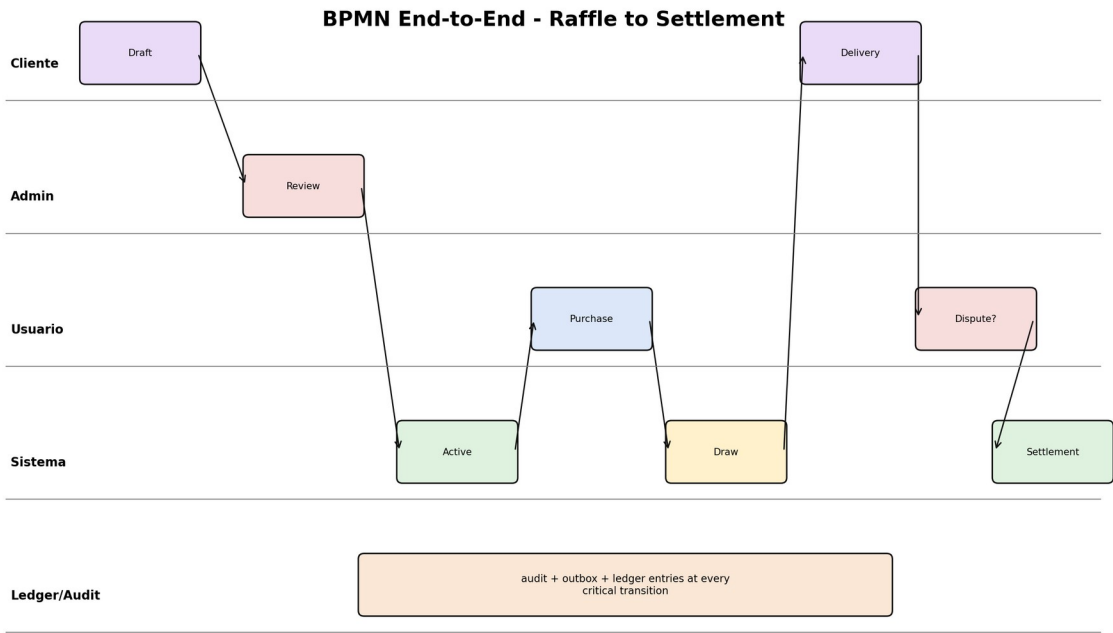


Figura 5. BPMN end-to-end de creación, compra, draw, delivery y settlement.

9. ERD maestro audit-ready

La base de datos debe ser audit-ready desde el diseño. No basta con logs de aplicación. El modelo debe tener tablas append-only, trace_id, payload_hash, state_transitions, event_outbox, ledger_entries y draw_proofs. Esto permite reconstrucción histórica, replay, conciliación y análisis forense.

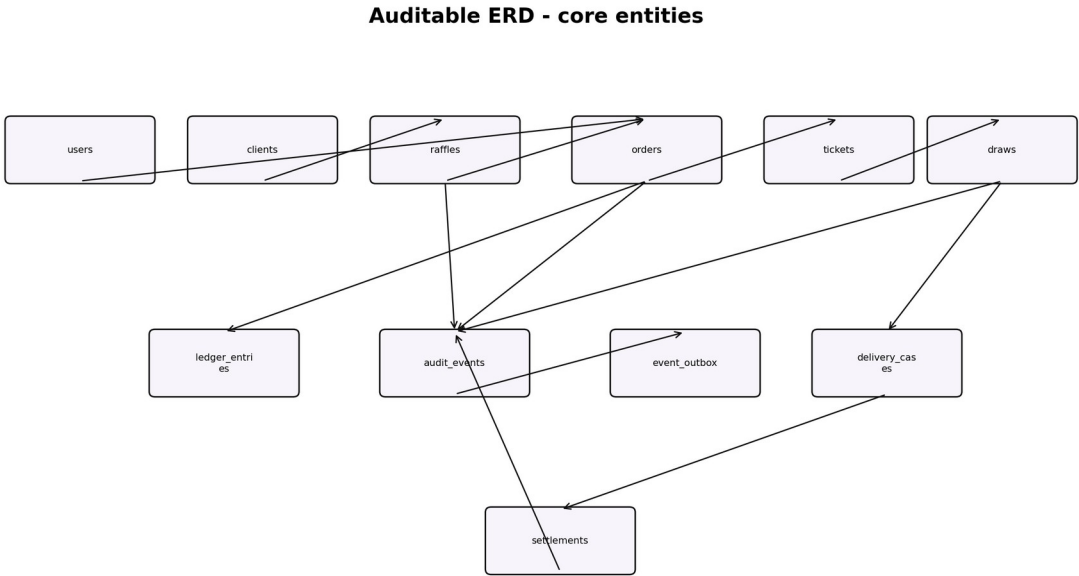


Figura 6. ERD maestro auditable.

Tabla	Propósito	Campos críticos
users	identidad usuario	user_id, email, status
clients	perfil cliente	client_id, legal_name, status
raffles	sorteos	raffle_id, client_id, type, status, pricing_config, draw_config
orders	compras	order_id, user_id, raffle_id, status, idempotency_key
tickets	unidad probabilística	ticket_id, raffle_id, order_id, ticket_number, status
draw_executions	ejecución sorteo	draw_id, raffle_id, mode, algorithm_version
draw_proofs	fairness	pool_hash, external_entropy, seed_material, random_value
ledger_entries	doble entrada	entry_id, account, debit, credit, reference_id
audit_events	auditoría fuerte	event_type, entity_id, actor, trace_id, payload_hash
event_outbox	publicación confiable	event_type, payload, status, retries

PARTE III - PURCHASE ENGINE

10. Propósito del Purchase Engine

Purchase convierte una intención de compra en una orden, una interacción PSP, tickets válidos, ledger y auditoría. Su regla principal es que no existe ticket válido sin pago consolidado y sin registro financiero asociado. Este módulo debe controlar idempotencia, PSP, locks, webhooks duplicados, recovery, reconcile y trazabilidad.

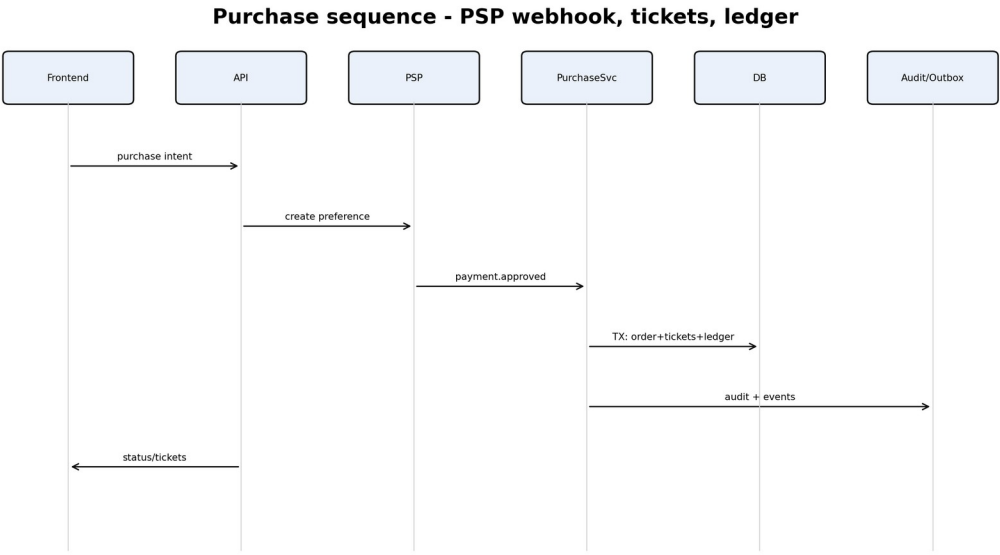


Figura 7. Sequence diagram del flujo Purchase con PSP, tickets y ledger.

11. APIs de Purchase

Método	Endpoint	Propósito	Auth	Audit event
POST	/v1/orders/intent	crear intención de compra	User	ORDER_INTENT_CREATED
POST	/v1/orders/purchase	iniciar checkout	User + Idempotency-Key	PAYMENT_INITIATED
GET	/v1/orders/{id}	consultar orden	User/Admin	ORDER_VIEWED
GET	/v1/orders/{id}/tickets	ver tickets emitidos	User/Admin	TICKETS_VIEWED
POST	/v1/payments/webhooks/mercadopago	webhook PSP	signed webhook	PAYMENT_WEBHOOK_RECEIVED
POST	/v1/internal/orders/{id}/reconcile	reconciliar orden	Service/Admin	ORDER_RECONCILED
POST	/v1/internal/orders/{id}/refund	refund operativo	Admin/Internal	REFUND_REQUESTED

```
POST /v1/orders/purchase
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid>
  X-Correlation-ID: <trace_id>
Request:
{
  "raffle_id": "rf_123",
  "quantity": 3,
  "payment_method": "mercadopago",
  "ticket_bundle_code": "BUNDLE_3"
}
Response 201:
{
  "order_id": "ord_987",
  "status": "PAYMENT_PENDING",
```

```

"payment_provider": "mercadopago",
"payment_reference": "mp_pref_123",
"next_action": {"type": "REDIRECT", "url": "https://..."},
"trace_id": "trc_abc"
}

```

12. Pseudocódigo transaccional

La creación de intención y la consolidación por webhook se separan. El sistema no emite tickets hasta recibir confirmación suficiente del PSP y pasar validaciones de estado, risk, idempotencia y disponibilidad.

```

def process_payment_approved(payment_id):
    if webhook_processed(payment_id):
        append_audit_event('WEBHOOK_DUPLICATE_IGNORED', 'payment', payment_id, SYSTEM, None, None, {})
        return
    payment = psp_get_payment(payment_id)
    order = find_order_by_payment(payment_id)
    begin_transaction()
    raffle = select_for_update('raffles', order.raffle_id)
    assert raffle.status == 'ACTIVE'
    assert raffle.available_tickets >= order.qty
    mark_order_paid(order.id, payment_id)
    create_ledger_entry('PURCHASE_DEBIT', order.amount, order.id)
    for i in range(order.qty):
        ticket_number = next_ticket_number(raffle.id)
        create_ticket(order.id, raffle.id, order.user_id, ticket_number)
    append_audit_event('PURCHASE_COMPLETED', 'order', order.id, SYSTEM, 'PAYMENT_PENDING', 'PAID',
{'payment_id': payment_id})
    create_outbox('TicketPurchased', order.id)
    commit_transaction()

```

13. Concurrencia, retries y edge cases

Riesgo	Escenario	Mitigación	Auditoría
Doble click	usuario reintenta request	Idempotency-Key + stored response	PURCHASE_RETRY_DETECT ED
Webhook duplicado	PSP envía dos eventos	processed_webhooks UNIQUE(payment_id)	WEBHOOK_DUPLICATE_IGNORED
Sobreventa	últimos tickets concurrentes	SELECT FOR UPDATE + UNIQUE(raffle_id,ticket_number)	TICKET_ISSUED
Pago aprobado sin commit	caída backend post PSP	recovery_job + reconcile_daily	RECOVERY_ORDER_FIXED
Timeout PSP	checkout no confirma	retry + pending aging	PAYMENT_TIMEOUT_MONITORING

PARTE IV - DRAW ENGINE

14. Propósito y fairness

Draw convierte tickets válidos en ganador verificable. Debe congelar el pool, serializar de forma canónica, calcular pool_hash, obtener entropía externa verificable, derivar seed_material, calcular random_value y resolver winner_index. Todos los insumos deben persistirse para verificación externa y auditoría.

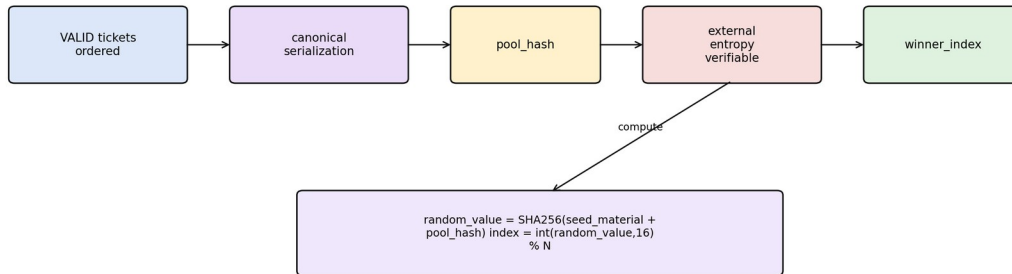
Draw fairness - cryptographic proof

Figura 8. Fairness verificable del Draw Engine.

Elemento	Función	Auditoría
canonical_pool	lista ordenada de tickets válidos	POOL_FROZEN
pool_hash	resumen criptográfico del pool	POOL_HASH_CREATED
external_entropy	fuentes externa verificable	EXTERNAL_ENTROPY_FETCHED
seed_material	material derivado	SEED_MATERIAL_CREATED
random_value	valor de cálculo final	RANDOM_VALUE_CREATED
certificate	evidencia formal	DRAW_CERTIFIED

```

def execute_draw(raffle_id, mode):
    acquire_lock(raffle_id)
    raffle = get_raffle(raffle_id)
    assert raffle.status == 'READY'
    pool = fetch_valid_tickets_ordered(raffle_id)
    canonical_pool = canonicalize(pool)
    pool_hash = sha256(serialized(canonical_pool))
    external_entropy = get_external_entropy() # public/verifiable source
    seed_material = sha256(pool_hash + external_entropy + raffle_id)
    random_value = sha256(seed_material + pool_hash)
    winner_index = int(random_value, 16) % len(canonical_pool)
    winner_ticket = canonical_pool[winner_index]
    persist_draw_execution(...)
    persist_draw_proof(...)
    append_audit_event('DRAW_EXECUTED', 'draw', draw_id, actor_from_mode(mode), 'READY', 'EXECUTED',
    {'winner_ticket_id': winner_ticket.id})
    publish_outbox('DrawExecuted', draw_id)
    release_lock(raffle_id)
  
```

15. LIVE execution y anti-manipulación

El modo T8 LIVE permite que el Cliente ejecute visualmente el sorteo solo si existe autorización Admin previa. La ejecución LIVE nunca modifica el algoritmo. El sistema debe capturar LIVE_AUTHORIZED, LIVE_TRIGGERED, POOL_FROZEN y DRAW_EXECUTED en la misma cadena de auditoría. Cualquier intento LIVE sin autorización debe rechazarse y auditarse como LIVE_TRIGGER_REJECTED.

PARTE V - DELIVERY + SETTLEMENT

16. Delivery lifecycle

Delivery inicia cuando Draw produce un ganador. El sistema abre delivery_case, notifica al Cliente para subir evidencia, solicita confirmación al Usuario y bloquea Settlement hasta que exista confirmación, timeout resuelto o disputa cerrada. Este módulo protege el cumplimiento real del premio y evita liquidaciones prematuras.

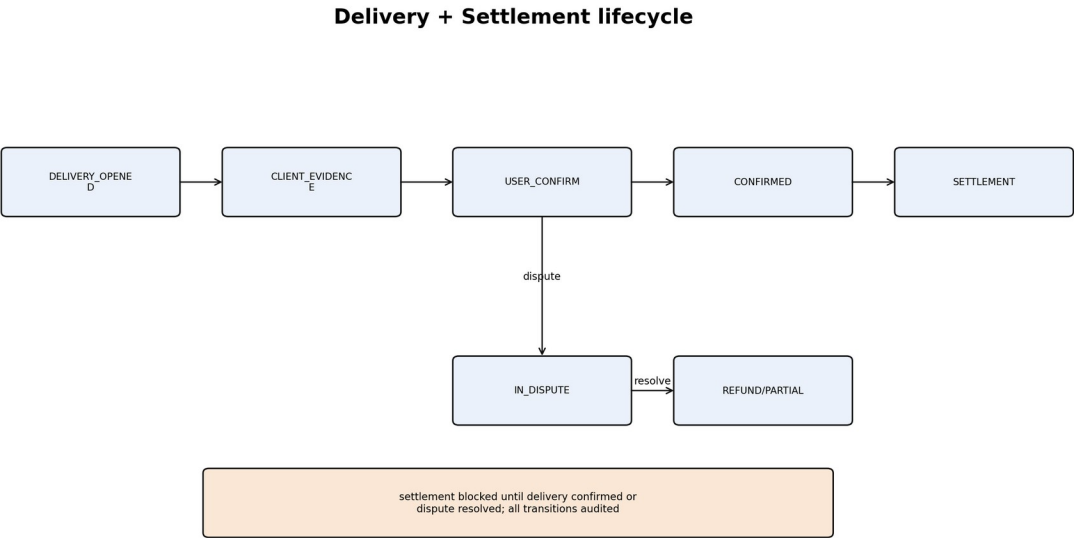


Figura 9. Delivery + Settlement lifecycle integrado.

Estado	Descripción	Actor	Auditoría
DELIVERY_OPENED	caso creado tras draw	Sistema	DELIVERY_OPENED
PENDING_CLIENT_EVIDENCE	cliente debe cargar evidencia	Cliente	DELIVERY_EVIDENCE_UPLOADED
PENDING_USER_CONFIRMATION	usuario valida tenencia	Usuario	DELIVERY_CONFIRM_REQUESTED
CONFIRMED	usuario confirma entrega	Usuario/Sistema	DELIVERY_CONFIRMED
IN_DISPUTE	reclamo o bloqueo	Usuario/Admin	DISPUTE_OPENED
RESOLVED	admin resuelve	Admin	DISPUTE_RESOLVED

```
def open_delivery_case(draw_id):
    winner = get_winner(draw_id)
    case = create_delivery_case(draw_id, winner.user_id, status='PENDING_CLIENT_EVIDENCE')
    append_audit_event('DELIVERY_OPENED', 'delivery_case', case.id, SYSTEM, None,
        'PENDING_CLIENT_EVIDENCE', {'draw_id': draw_id})
    notify_client(case)

def upload_delivery_evidence(client_id, case_id, evidence):
    assert client_owns_raffle(client_id, case_id)
    save_evidence(case_id, evidence)
    transition(case_id, 'PENDING_USER_CONFIRMATION')
    append_audit_event('DELIVERY_EVIDENCE_UPLOADED', 'delivery_case', case_id, CLIENT,
        'PENDING_CLIENT_EVIDENCE', 'PENDING_USER_CONFIRMATION', {'files': evidence.refs})
```

17. Settlement lifecycle y contabilidad

Settlement libera obligaciones económicas después de Delivery. Si existe disputa abierta, el settlement queda congelado. Si el Usuario confirma, se habilita split/payout. Si Admin resuelve a favor del Usuario, se activa refund total o parcial. El diseño es escrow-ready: aunque el MVP use operación simplificada, los estados y ledger deben soportar retención, liberación y reverso.

Ledger model - double entry and settlement

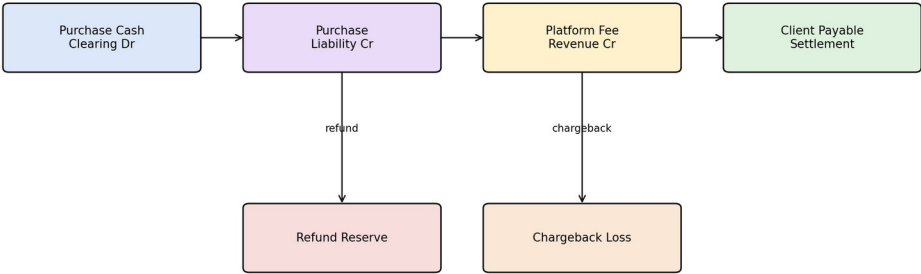


Figura 10. Modelo ledger doble entrada y settlement.

Caso	Debit	Credit	Condición
Cliente confirmado	Purchase Liability	Client Payable	delivery confirmed
Fee ALAZAR	Purchase Liability	Platform Revenue	según política
Disputa usuario	Settlement Freeze	Purchase Liability	dispute open
Refund usuario	Refund Reserve	User Wallet / PSP reverse	resolución usuario
Settlement parcial	Purchase Liability	Client Payable parcial	resolución admin

```
def trigger_settlement(case_id):
    case = get_delivery_case(case_id)
    assert case.status in ['CONFIRMED', 'RESOLVED_CLIENT']
    assert no_open_dispute(case_id)
    begin_transaction()
    settlement = create_settlement(case.raffle_id, status='PROCESSING')
    create_double_entries_for_settlement(settlement)
    call_psp_split_or_mark_manual(settlement)
    append_audit_event('SETTLEMENT_TRIGGERED', 'settlement', settlement.id, SYSTEM, None, 'PROCESSING',
{'case_id': case_id})
    commit_transaction()
```

18. Dispute orchestration

La disputa es un mecanismo de protección de confianza. Cuando el Usuario cuestiona entrega, calidad, tenencia o cumplimiento, el sistema congela Settlement y abre dispute_case. El Admin revisa evidencia del Cliente, evidencia del Usuario, proof del Draw, orden de compra y ledger. La resolución debe incluir motivo, efecto financiero, actor, timestamp y audit_event.

Estado disputa	Descripción	Resultado posible
OPEN	disputa creada	freeze settlement
UNDER_REVIEW	admin revisa evidencia	solicitar datos
RESOLVED_CLIENT	a favor cliente	habilita settlement
RESOLVED_USER	a favor usuario	refund total/parcial
CLOSED	caso cerrado	audit final

PARTE VI - AUDITORÍA Y COMPLIANCE

19. Modelo de auditoría fuerte

La auditoría es transversal y append-only. Cada acción que cambie dinero, tickets, estado, ganador, permisos o payout debe generar audit_event. El audit_event debe registrar actor, entidad, estado previo, estado nuevo, payload_hash, metadata y trace_id. El trace_id conecta API, DB, logs, webhooks, outbox, ledger y workers.

Campo	Propósito
event_id	identificador único del evento
trace_id	hilo de trazabilidad end-to-end
event_type	tipo de evento de dominio
entity_type/entity_id	objeto afectado
actor_type/actor_id	quién o qué ejecutó
previous_state/new_state	transición observable
payload_hash	integridad del payload canónico
metadata_json	contexto operativo
event_version	compatibilidad histórica

```
def append_audit_event(event_type, entity_type, entity_id, actor, prev_state, new_state, payload):
    canonical_payload = canonical_json(payload)
    payload_hash = sha256(canonical_payload)
    audit_events.insert({
        'event_type': event_type,
        'entity_type': entity_type,
        'entity_id': entity_id,
        'actor_type': actor.type,
        'actor_id': actor.id,
        'trace_id': current_trace_id(),
        'previous_state': prev_state,
        'new_state': new_state,
        'payload_hash': payload_hash,
        'metadata_json': payload,
        'event_version': 'v1',
        'occurred_at': now()
    })
```

20. Forensic reconstruction y replay

El replay permite reconstruir qué ocurrió con una compra, sorteo, entrega o settlement. No debe usarse como único método de estado operacional, pero sí como evidencia de auditoría y herramienta forense. Para un trace_id, el sistema debe mostrar timeline de acciones, actores, payload_hash y referencias cruzadas a ledger, tickets, draw_proofs y webhooks.

Dominio	Evento clave	Uso forense
Raffle	RAFFLE_APPROVED	demostrar que Admin autorizó publicación
Purchase	PURCHASE_COMPLETED	probar pago y tickets
Draw	POOL_FROZEN / DRAW_EXECUTED	demostrar fairness
Delivery	DELIVERY_EVIDENCE_UPLOADED	probar cumplimiento
Dispute	DISPUTE_RESOLVED	justificar payout/refund
Settlement	SETTLEMENT_EXECUTED	cerrar obligación financiera

PARTE VII - APIs ENTERPRISE

21. Estándares API enterprise

Las APIs deben estar documentadas estilo OpenAPI: endpoints, métodos, auth, headers, request, response, errores, idempotencia, rate limits, firmas y trace propagation. Las APIs que producen efectos requieren Idempotency-Key. Los endpoints Admin requieren MFA, scopes y reason_required si cambian estados sensibles.

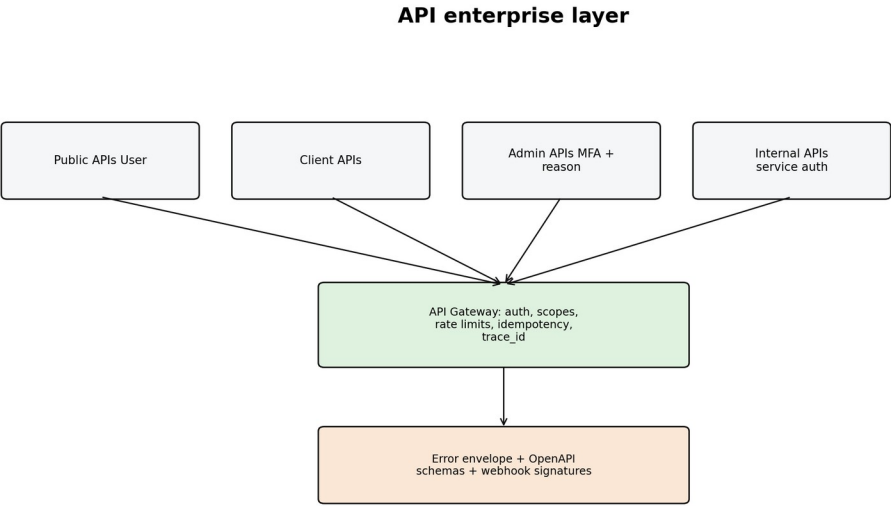


Figura 11. Capa API enterprise con auth, rate limits, idempotencia y trace_id.

Header	Obligatorio	Uso
Authorization	sí	JWT / service token
Idempotency-Key	sí en comandos	evitar efectos duplicados
X-Correlation-ID	recomendado/sí interno	propagar trace_id
X-Webhook-Signature	sí webhooks	validar origen PSP
X-RateLimit-*	respuesta	control abuso y backpressure

```
Error envelope standard:
{
  "error_code": "ERR_PURCHASE_003",
  "message": "Not enough tickets available",
  "trace_id": "trc_abc123",
  "retryable": false,
  "details": {"raffle_id": "rf_123", "available": 2, "requested": 3}
}
```

Familia API	Ejemplos	Requisitos
Public User	/v1/orders/purchase, /v1/users/me/tickets	Bearer token, idempotency
Client	/v1/client/raffles, /v1/client/delivery/evidence	RBAC cliente, audit trail
Admin	/v1/admin/raffles, /v1/admin/disputes	MFA, reason_required
Internal Workers	/v1/internal/reconcile, /v1/internal/recover	service auth, trace_id
Webhooks	/v1/payments/webhooks/mercadopago	signature, replay protection

PARTE VIII - INFRAESTRUCTURA Y OBSERVABILIDAD

22. Topología de infraestructura

La infraestructura debe soportar operación fintech real: separación de ambientes, secretos, backups, workers, colas, DLQ, monitoreo, alertas y recuperación. Para MVP se puede operar en una nube única con arquitectura modular; el diseño debe permitir evolucionar a multi-zona, read replicas y procesos de disaster recovery.

Infrastructure topology - MVP to fintech scale

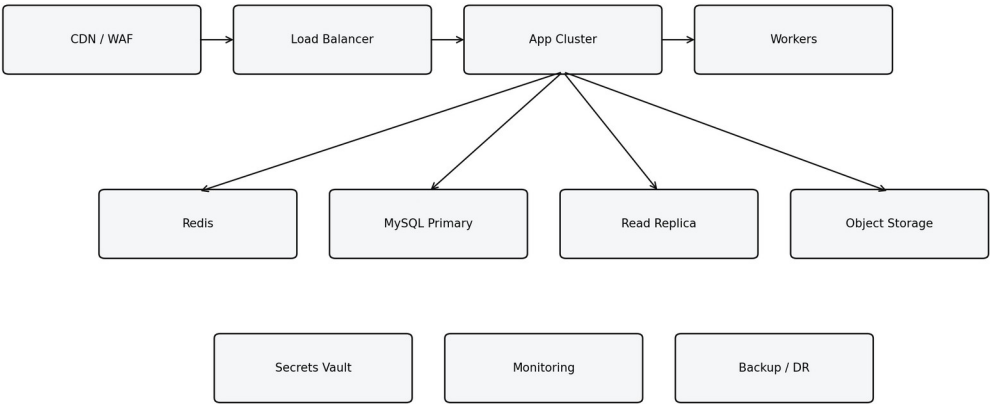


Figura 12. Topología de infraestructura MVP a escala fintech.

Componente	Política operacional
Outbox	publicación atómica con DB transaction
DLQ	eventos fallidos después de N retries
Backup	snapshots diarios + point-in-time recovery
Logs	JSON estructurado con trace_id
Secrets	vault y rotación
Deploy	blue/green con rollback
Autoscaling	basado en CPU, queue lag y latencia
Circuit breakers	proteger contra PSP o servicios degradados

23. Observabilidad SRE

Observabilidad no es solo logging. El sistema necesita logs estructurados, métricas, trazas distribuidas, dashboards por dominio, alertas y auditoría consultable. Un incidente de compra, draw o settlement debe poder investigarse por trace_id sin reconstrucción manual.

Observability - logs, metrics, traces, alerts

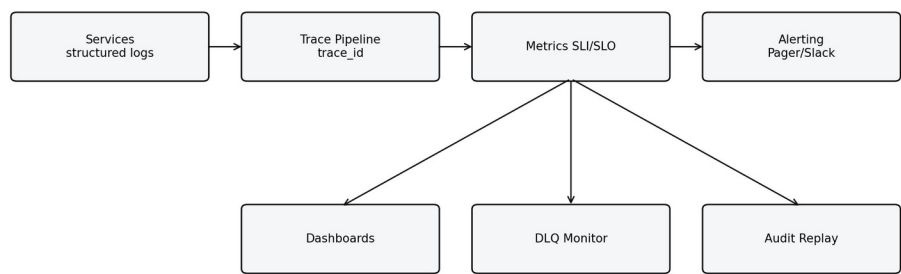


Figura 13. Observabilidad: logs, métricas, trazas, alertas y replay.

Indicador	Definición	Alerta
purchase_success_rate	órdenes completadas / intentos válidos	< 95% en 15 min
webhook_lag_ms	tiempo entre PSP approved y procesamiento	p95 > 60s
draw_execution_ms	latencia de sorteo	p95 > 5s
proof_verify_fail_rate	fallos de verificación draw	> 0
settlement_aging	settlements pendientes	> SLA
reconciliation_mismatch_rate	diferencias PSP vs ledger	> umbral

PARTE IX - QA, RESILIENCIA Y ROADMAP

24. Testing pyramid y resiliencia

La QA debe validar invariantes de negocio, no solo pantallas. Debe probarse que no existen tickets sin purchase debit, que un draw no corre sin pool frozen, que settlement se bloquea ante disputa y que cada acción crítica genera audit_event. Además, se requieren pruebas de carga, contract testing, replay testing y chaos engineering controlado.

Testing pyramid + resilience

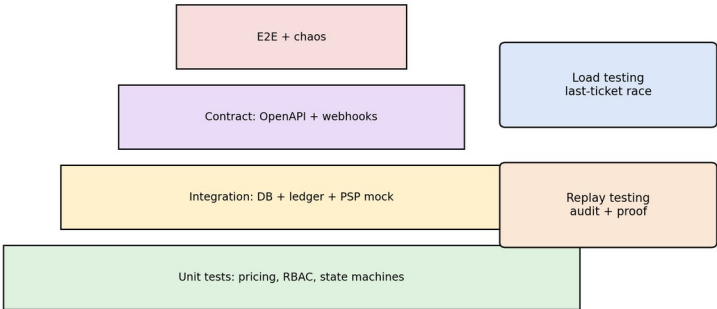


Figura 14. Testing pyramid y pruebas de resiliencia.

Tipo de prueba	Objetivo	Ejemplo
Unit	validar reglas puras	pricing engine, RBAC, state transitions
Integration	validar DB y servicios	purchase + ledger + tickets
Contract	validar APIs	OpenAPI request/response
Load	concurrency	últimos tickets simultáneos
Security	auth, RBAC, webhooks	firma PSP, replay protection
Audit	trazabilidad	replay por trace_id
Chaos	resiliencia	caída PSP, worker delay, DLQ

25. Roadmap de implementación

Fase	Alcance	Resultado
MVP-1	Purchase + Draw + auditoría base	tickets y ganador verificable
MVP-2	Delivery + Settlement + disputa	flujo económico cerrado
MVP-3	APIs enterprise + observabilidad	operación productiva
Post-MVP	split real / escrow / multi-país	escalabilidad fintech

Este PRD debe mantenerse vivo con versionado. Toda mejora futura debe agregarse como delta acumulativo, no como reescritura destructiva. Las decisiones de arquitectura, reglas de negocio, APIs, tablas y algoritmos deben conservar historial para evitar degradación de coherencia.

ANEXO A - Matriz de eventos auditables por dominio

Dominio	Evento	Trigger	Payload mínimo	Consumidores
Raffle	RAFFLE_CREATED	cliente crea draft	client_id, type, target	Audit, Admin
Raffle	RAFFLE_APPROVED	admin aprueba	admin_id, criteria	Audit, Notification
Pricing	PRICING_CALCULATED	cliente simula	target, fees, scenarios	Audit
Purchase	ORDER_INTENT_CREATED	usuario inicia compra	order_id, qty, amount	Audit, Risk
Purchase	PURCHASE_COMPLETED	PSP aprobado + commit	order_id, ticket_ids	Audit, Draw, Notify
Draw	POOL_FROZEN	draw inicia	pool_hash, count	Audit
Draw	DRAW_EXECUTED	winner calculado	seed, random_value, winner	Audit, Delivery
Delivery	DELIVERY_CONFIRMED	usuario confirma	case_id, timestamp	Audit, Settlement
Dispute	DISPUTE_OPENED	usuario reclama	reason, evidence	Audit, Admin
Settlement	SETTLEMENT_EXECUTED	payout/refund	ledger refs, amounts	Audit, Finance

ANEXO B - DDL lógico audit-ready

```
CREATE TABLE audit_events (
  event_id VARCHAR(64) PRIMARY KEY,
  trace_id VARCHAR(64) NOT NULL,
  event_type VARCHAR(64) NOT NULL,
  entity_type VARCHAR(64) NOT NULL,
  entity_id VARCHAR(64) NOT NULL,
  actor_type VARCHAR(32) NOT NULL,
  actor_id VARCHAR(64),
  previous_state JSON,
  new_state JSON,
  payload_hash VARCHAR(128) NOT NULL,
  metadata_json JSON NOT NULL,
  event_version VARCHAR(16) NOT NULL,
  occurred_at TIMESTAMP NOT NULL,
  immutable_flag BOOLEAN DEFAULT TRUE
);
```

```
CREATE TABLE tickets (
  ticket_id VARCHAR(64) PRIMARY KEY,
  raffle_id VARCHAR(64) NOT NULL,
  order_id VARCHAR(64) NOT NULL,
  user_id VARCHAR(64) NOT NULL,
  ticket_number INT NOT NULL,
  status VARCHAR(32) NOT NULL,
  created_at TIMESTAMP NOT NULL,
  UNIQUE(affle_id, ticket_number)
);
```

```
CREATE TABLE draw_proofs (
  proof_id VARCHAR(64) PRIMARY KEY,
  draw_id VARCHAR(64) NOT NULL,
  pool_hash VARCHAR(128) NOT NULL,
  external_entropy VARCHAR(256) NOT NULL,
  seed_material VARCHAR(128) NOT NULL,
  random_value VARCHAR(128) NOT NULL,
  algorithm_version VARCHAR(32) NOT NULL,
  created_at TIMESTAMP NOT NULL
);
```

ANEXO C - Invariantes no negociables

Invariante	Validación	Severidad
------------	------------	-----------

No existe ticket válido sin order PAID	DB + test integration	Crítica
No existe draw sin POOL_FROZEN	state machine + audit	Crítica
No existe settlement con dispute abierta	settlement guard	Crítica
Todo override Admin requiere motivo	RBAC + audit	Alta
Todo webhook es idempotente	processed_webhooks UNIQUE	Crítica
Todo proof de draw es reproducible	verify_draw()	Crítica
Ledger debe cuadrar debit = credit	reconcile job	Crítica

ANEXO D - OpenAPI-style contracts enterprise

D.1 Estándar de contrato por endpoint

Cada endpoint del PRD debe especificarse con método, ruta, actor autorizado, scopes, headers, request schema, response schema, errores, idempotencia, eventos emitidos, métricas y auditoría generada. Este estándar evita que frontend, backend, QA y operaciones interpreten de manera distinta el comportamiento esperado de una misma API.

Campo de contrato	Descripción	Obligatorio
method + path	verbo HTTP y ruta versionada	Sí
actor/scopes	rol y permisos requeridos	Sí
headers	Authorization, Idempotency-Key, X-Correlation-ID	Sí en comandos
request schema	payload esperado con tipos y restricciones	Sí
response schema	estructura de respuesta de éxito	Sí
error envelope	códigos de error normalizados	Sí
audit event	evento generado si hay cambio sensible	Sí
metrics	métricas operativas del endpoint	Sí para endpoints críticos

```
POST /v1/draw/execute
Auth: Bearer token, scope=draw.execute
Headers:
  Idempotency-Key: required
  X-Correlation-ID: required
Request:
{
  "raffle_id": "rf_123",
  "mode": "AUTO|LIVE|ADMIN",
  "requested_winners": 1
}
Response 201:
{
  "draw_id": "dr_001",
  "status": "EXECUTED",
  "winner_ticket_id": "tck_901",
  "proof_url": "/v1/draw/dr_001/proof",
  "trace_id": "trc_123"
}
Audit: DRAW_REQUESTED, POOL_FROZEN, DRAW_EXECUTED
Metrics: draw_execution_ms, proof_generation_ms, draw_error_rate
```

D.2 Catálogo API extendido por dominio

El catálogo API del producto debe cubrir flujos públicos, cliente, admin, internos y webhooks. La separación por familia permite aplicar políticas diferentes de autenticación, rate limit, auditoría y monitoreo. Los endpoints internos no deben ser accesibles desde internet pública y deben usar service-to-service auth.

Dominio	Endpoint	Actor	Idempotencia	Audit event
Raffle	POST /v1/raffles	Cliente	Sí	RAFFLE_CREATED
Raffle	POST /v1/raffles/{id}/submit	Cliente	Sí	RAFFLE_SUBMITTED
Admin	POST /v1/admin/raffles/{id}/approve	Admin	Sí	RAFFLE_APPROVED
Pricing	POST /v1/raffles/{id}/pricing/simulate	Cliente	No	PRICING_CALCULATED
Purchase	POST /v1/orders/purchase	Usuario	Sí	ORDER_INTENT_CREATED
Payments	POST /v1/payments/webhooks/mercadopago	PSP	Sí por payment_id	PAYMENT_WEBHOOK_RECEIVED
Tickets	GET /v1/orders/{id}/tickets	Usuario/Admin	No	TICKETS_VIEWED
Draw	POST /v1/draw/execute	System/Admin/Cliente autorizado	Sí	DRAW_REQUESTED
Delivery	POST	Cliente	Sí	DELIVERY_EVIDENCE_

	/v1/delivery/{id}/evidence			UPLOADED
Dispute	POST /v1/disputes	Usuario/Admin	Sí	DISPUTE_OPENED
Settlement	POST /v1/settlements/{id}/execute	System/Admin	Sí	SETTLEMENT_EXECUTED
Audit	GET /v1/admin/audit?trace_id=	Admin	No	AUDIT_VIEWED

ANEXO E - Ledger bank-grade expansion

E.1 Chart of accounts lógico

El ledger de ALAZAR no debe limitarse a un log de pagos. Debe modelar cuentas lógicas que permitan reconocer obligaciones, ingresos, costos, reservas, refunds, chargebacks y pagos a clientes. Esta estructura permite evolucionar de MVP a operación con split real o escrow sin rediseñar la contabilidad interna.

Cuenta lógica	Tipo	Uso
Cash Clearing	Asset	dinero capturado por PSP pendiente de conciliación
User Wallet / User Receivable	Liability/Asset	posición del usuario según modelo operativo
Purchase Liability	Liability	obligación generada por tickets vendidos hasta settlement
Platform Revenue	Revenue	fee ALAZAR reconocido según política
Payment Expense	Expense	costo PSP
Client Payable	Liability	monto neto a pagar al cliente
Refund Reserve	Liability	reserva para devoluciones
Chargeback Loss	Expense	pérdidas por contracargos
Settlement Freeze	Contra-liability	bloqueo temporal por disputa

E.2 Journal entries por caso

Caso	Debit	Credit	Notas
Purchase approved	Cash Clearing	Purchase Liability	captura de dinero y obligación
Platform fee recognition	Purchase Liability	Platform Revenue	fee del 20% según política
PSP fee	Payment Expense	Cash Clearing	costo de pasarela
Client settlement	Purchase Liability	Client Payable / Cash	pago al cliente tras delivery
Full refund	Refund Reserve	Cash / PSP Reverse	reverso total
Partial refund	Refund Reserve parcial	Cash / PSP Reverse parcial	por decisión Admin
Chargeback	Chargeback Loss	Cash Clearing	contracargo confirmado
Dispute freeze	Settlement Freeze	Purchase Liability	bloqueo temporal

```
def post_journal(event):
    entries = accounting_policy.resolve(event)
    assert sum(e.debit for e in entries) == sum(e.credit for e in entries)
    for e in entries:
        ledger_entries.insert(e.with_trace_id(current_trace_id()))
    append_audit_event('LEDGER_POSTED', 'ledger_batch', event.reference_id, SYSTEM, None, None,
{'entry_count': len(entries)})
```

ANEXO F - Delivery, dispute y settlement profundo

F.1 Evidence lifecycle

La evidencia de entrega debe ser tratada como objeto de cumplimiento. Puede incluir fotografías, documentos, firma digital, constancia de servicio o evidencia logística. Cada evidencia debe registrar uploader, timestamp, hash del archivo, tipo, relación con delivery_case y estado de validación. La evidencia no resuelve automáticamente la entrega; habilita al usuario a confirmar o disputar.

Evidencia	Actor	Validaciones	Auditoría
foto de entrega	Cliente	formato, tamaño, hash	DELIVERY_EVIDENCE_UPLOADED
constancia de servicio	Cliente	documento válido	DELIVERY_DOCUMENT_UPLOADED
confirmación de tenencia	Usuario	ownership del caso	DELIVERY_CONFIRMED
reclamo usuario	Usuario	ventana activa	DISPUTE_OPENED
resolución admin	Admin	motivo obligatorio	DISPUTE_RESOLVED

F.2 Settlement gates

Settlement debe estar gobernado por gates. Un gate es una condición obligatoria que impide liberar dinero si no se cumple. Los gates mínimos son: Draw ejecutado, Delivery abierto, evidencia cargada o timeout aplicable, no disputa abierta, ledger balanceado y conciliación PSP sin mismatch crítico.

Gate	Condición	Si falla
DRAW_EXECUTED	existe ganador y proof	no abrir settlement
DELIVERY_RESOLVED	confirmado o resolución admin	mantener bloqueado
NO_OPEN_DISPUTE	no hay disputa abierta	freeze payout
LEDGER_BALANCED	debit = credit	crear caso financiero
PSP_RECONCILED	monto PSP coincide	reconciliation case
ADMIN_OVERRIDE_REASON	si hay override existe motivo	rechazar acción

```
def can_execute_settlement(raffle_id):
    assert draw_exists(raffle_id)
    assert delivery_case_resolved(raffle_id)
    assert no_open_dispute(raffle_id)
    assert ledger_balanced_for_raffle(raffle_id)
    assert psp_reconciled_or_admin_override(raffle_id)
    return True
```

ANEXO G - Risk, abuso y threat model

G.1 Risk model operacional

Risk opera antes, durante y después de la compra. Evalúa velocidad de compras, concentración de tickets, IP/device, historial de disputas, chargebacks, clientes con patrones anómalos y draws con comportamiento inusual. Sus decisiones deben ser explicables y auditadas para evitar cajas negras operativas.

Señal	Riesgo	Acción	Evento
velocity alta	bot / abuso	throttle	RISK_THROTTLE_APPLIED
device compartido	multi-accounting	score penalty	RISK_SCORE_UPDATED
chargeback previo	pérdida financiera	freeze wallet	WALLET_FROZEN
cliente con disputas	riesgo reputacional	manual review	CLIENT_REVIEW_REQUIRED
concentración tickets	manipulación percibida	alerta admin	RAFFLE_CLUSTER_ALERT
LIVE fuera de patrón	abuso operacional	bloquear trigger	LIVE_TRIGGER_BLOCKED

G.2 Threat model mínimo

Amenaza	Vector	Control
Webhook spoofing	falso evento PSP	firma, correlation, PSP lookup
Replay de webhook	reenvío de evento válido	processed_webhooks UNIQUE
Doble ejecución Draw	reintento concurrente	lock + executed flag
Manipulación Admin	override sin control	reason_required + audit
Ticket fraud	creación fuera de purchase	FK + invariant tests
API abuse	alta frecuencia	rate limit + risk score
Data tampering	modificación audit log	append-only + payload_hash

ANEXO H - Observability runbooks y SLOs

H.1 SLOs por dominio

Los SLOs deben reflejar experiencia y seguridad operacional. No basta medir disponibilidad global: se deben medir flujos críticos como compra, webhook, draw, proof verification, settlement y audit append. Cada SLO debe tener alerta, dashboard y runbook.

SLO	Objetivo	Alerta	Runbook
purchase_success_rate	>= 98%	<95% 15min	ver PSP, DB locks, error rate
webhook_processing_lag	p95 < 60s	p95 > 60s	revisar queue, PSP, workers
draw_execution_success	99.9%	fallo >0 crítico	bloquear settlement y revisar proof
audit_append_success	99.99%	cualquier caída	modo degradado: bloquear acciones sensibles
settlement_execution_time	p95 < 5min	aging > SLA	revisar gates, PSP payout
reconciliation_mismatch	casi 0	> umbral	abrir finance case

H.2 Runbook ejemplo: webhook lag

Runbook: `webhook_processing_lag` alto

1. Verificar queue depth de `payment_webhook_events`.
2. Verificar latencia de DB y locks en orders.
3. Confirmar disponibilidad PSP lookup API.
4. Escalar workers si CPU/queue lag lo exige.
5. Si hay duplicados masivos, activar filtro por `processed_webhooks`.
6. Abrir incident log y registrar `AUDIT_OPERATIONAL_INCIDENT`.
7. Ejecutar `reconcile_daily` si el lag superó SLA.

ANEXO I - QA enterprise y criterios de aceptación ampliados

I.1 Matriz de pruebas críticas

Caso	Tipo	Criterio de aceptación	Audit esperado
Compra exitosa genera tickets	Integration	order PAID, tickets VALID, ledger posted	PURCHASE_COMPLETED
Webhook duplicado	Integration	no duplica tickets ni ledger	WEBHOOK_DUPLICATE_IGNORED
Últimos tickets simultáneos	Load/concurrency	no sobreventa	TICKET_ISSUED range consistente
Draw estándar	Integration	winner reproducible	DRAW_EXECUTED
Draw LIVE sin autorización	Security	rechazo 403/409	LIVE_TRIGGER_REJECTED
Disputa bloquea settlement	Process	payout no ejecuta	SETTLEMENT_BLOCKED
Audit replay	Audit	timeline completo por trace_id	AUDIT_REPLAY_EXECUTED
PSP mismatch	Finance	case abierto	RECONCILIATION_CASE_OPENED

I.2 Property-based fairness testing

El Draw Engine debe probarse con property-based testing: distintos tamaños de pool, múltiples ordenamientos de entrada, múltiples semillas externas, multi-ganador sin reemplazo y reintentos. La propiedad principal es que, dado el mismo canonical_pool, external_entropy y algorithm_version, el resultado debe ser idéntico.

```
property test_draw_is_deterministic(pool, entropy):
    canonical = canonicalize(pool)
    r1 = execute_algorithm(canonical, entropy, algorithm_version='draw-v1')
    r2 = execute_algorithm(shuffle(pool), entropy, algorithm_version='draw-v1')
    assert r1.winner_ticket_id == r2.winner_ticket_id
    assert r1.pool_hash == r2.pool_hash
```

ANEXO D - Especificación enterprise extendida

Este anexo profundiza las áreas que elevan el PRD desde un documento funcional hacia un blueprint enterprise. Las siguientes secciones no sustituyen los capítulos anteriores; agregan detalle de implementación, gobierno, validación, operación y escalabilidad. Se mantienen los principios de auditoría transversal, doble entrada, event lineage y prueba reproducible del sorteo.

D.1 OpenAPI-style por dominio

Cada endpoint debe documentarse con actor, scopes, headers, request/response, errores, eventos emitidos, métricas y efectos de auditoría. Esto permite contract testing y evita ambigüedad entre frontend, backend y QA. Los comandos que mutan estado requieren Idempotency-Key; las consultas sensibles requieren audit de acceso si exponen datos financieros, evidencias o trazas internas.

Dominio	Endpoint	Actor	Headers críticos	Audit/Metric
Raffle	POST /v1/raffles	Cliente	Authorization, X-Correlation-ID	RAFFLE_CREATED / raffle_create_ms
Pricing	POST /v1/raffles/{id}/pricing/simulate	Cliente	Authorization, X-Correlation-ID	PRICING_CALCULATED / pricing_ms
Purchase	POST /v1/orders/purchase	Usuario	Authorization, Idempotency-Key, X-Correlation-ID	ORDER_INTENT_CREATED / purchase_intent_ms
Payments	POST /v1/payments/webhooks/mercadopago	PSP	X-Webhook-Signature	PAYMENT_WEBHOOK_RECEIVED / webhook_lag_ms
Draw	POST /v1/draw/execute	Admin/System/Cliente autorizado	Authorization, Idempotency-Key	DRAW_REQUESTED / draw_execution_ms
Delivery	POST /v1/delivery/{id}/evidence	Cliente	Authorization, X-Correlation-ID	DELIVERY_EVIDENCE_UPLOADED
Dispute	POST /v1/disputes	Usuario/Admin	Authorization, Idempotency-Key	DISPUTE_OPENED
Settlement	POST /v1/settlements/{id}/execute	System/Admin	Service-Auth, Idempotency-Key	SETTLEMENT_EXECUTE_D / settlement_ms
Audit	GET /v1/admin/audit?trace_id=	Admin	Authorization, MFA	AUDIT_VIEWED / audit_query_ms

```
# API command middleware
validate_auth(request)
validate_scope(actor, request.endpoint)
if request.is_command:
    require_header('Idempotency-Key')
trace_id = request.header('X-Correlation-ID') or generate_trace_id()
log_structured('API_COMMAND_RECEIVED', trace_id=trace_id, actor=actor.id, endpoint=request.path)
```

D.2 Contratos de error y retry

El modelo de error debe distinguir errores recuperables y no recuperables. Esto es clave para que frontend, workers y backoffice no reintenten operaciones que podrían duplicar efectos. Los errores de validación son no retryable; errores PSP o locks temporales pueden ser retryable con backoff controlado.

Código	HTTP	Dominio	Retryable	Acción
ERR_PURCHASE_003	409	Purchase	No	tickets insuficientes; refrescar disponibilidad
ERR_PURCHASE_005	502	PSP	Sí	retry con backoff y reconcile
ERR_DRAW_002	409	Draw	No	draw ya ejecutado; devolver resultado previo
ERR_DRAW_003	423	Draw	Sí	lock activo; retry seguro
ERR_SETTLEMENT_004	409	Settlement	No	disputa abierta; bloquear payout
ERR_AUDIT_001	500	Audit	No	bloquear acciones sensibles si audit no escribe

D.3 Webhooks, replay protection y reconciliación

Los webhooks externos deben considerarse no confiables hasta verificarse. El sistema valida firma, registra payload crudo en `webhook_events`, consulta el PSP como fuente autoritativa, aplica idempotencia por `payment_id/event_id` y luego ejecuta el handler de dominio. Si existe mismatch, se abre `reconciliation_case`.

Failure mode	Detección	Acción	Evento
Webhook duplicado	processed_webhooks hit	return 200 no-op	WEBHOOK_DUPLICATE_IGNORED
Firma inválida	signature check fail	reject 401	WEBHOOK_SIGNATURE_INVALID
Payload vs PSP mismatch	PSP lookup difiere	open case	WEBHOOK_MISMATCH_DETECTED
Webhook tardío	order aging	process/recover	WEBHOOK_LATE_PROCESSED
PSP timeout	lookup no responde	retry schedule	PSP_LOOKUP_RETRY

```
def webhook_handler(payload, headers):
    verify_signature(headers, payload)
    event_id = payload['id']
    if processed(event_id): return ok()
    payment = psp_lookup(payload['payment_id'])
    if inconsistent(payload, payment):
        create_reconciliation_case(payload, payment)
        append_audit_event('WEBHOOK_MISMATCH_DETECTED', ...)
        return accepted()
    route_payment_status(payment)
    mark_processed(event_id)
```

D.4 Ledger bank-grade y chart of accounts

El ledger debe permitir explicar todas las posiciones económicas del sistema: dinero capturado por PSP, obligaciones por tickets, revenue de plataforma, costo PSP, payable al cliente, refunds, chargebacks y reservas. Aunque el MVP pueda operar con un modelo simplificado, las cuentas lógicas deben existir desde el diseño para que el sistema sea escrow-ready y multi-país en el futuro.

Cuenta lógica	Tipo	Uso	Eventos
Cash Clearing	Asset	captura por PSP pendiente de conciliación	PURCHASE_APPROVED
Purchase Liability	Liability	obligación por tickets hasta settlement	PURCHASE_COMPLETED
Platform Revenue	Revenue	fee ALAZAR reconocido	FEE_RECOGNIZED
Payment Expense	Expense	costo PSP	PSP_FEE_RECORDED
Client Payable	Liability	monto neto al cliente	SETTLEMENT_TRIGGERED
Refund Reserve	Liability	reserva para devoluciones	REFUND_REQUESTED
Chargeback Loss	Expense	pérdidas por contracargo	CHARGEBACK_RECORDED

```
def post_ledger_batch(reference_id, entries):
    assert sum(e.debit for e in entries) == sum(e.credit for e in entries)
    batch_id = create_ledger_batch(reference_id)
    for entry in entries:
        ledger_entries.insert(entry.with_batch(batch_id).with_trace(current_trace_id()))
    append_audit_event('LEDGER_BATCH_POSTED', 'ledger_batch', batch_id, SYSTEM, None, None,
        {'reference_id': reference_id})
```

D.5 Delivery, dispute y settlement profundo

Delivery y Settlement deben operar como un único ciclo económico. Draw define ganador, Delivery prueba cumplimiento, Dispute protege al usuario y Settlement libera valor. El sistema debe impedir que payout ocurra si hay disputa abierta, si falta evidencia, si el ledger no cuadra o si PSP no está conciliado.

Gate	Condición	Si falla	Evento
DRAW_EXECUTED	winner y proof existen	no abrir settlement	SETTLEMENT_GATE_FAILED
DELIVERY_RESOLVED	confirmado o resolución admin	mantener bloqueado	DELIVERY_PENDING_BLOCK
NO_OPEN_DISPUTE	no hay disputa abierta	freeze payout	SETTLEMENT_BLOCKED
LEDGER_BALANCED	debit = credit	abrir finance case	LEDGER_MISMATCH
PSP_RECONCILED	PSP vs interno coincide	reconciliation case	RECONCILIATION_CASE_OPENED
ADMIN_REASON	override con motivo	rechazar acción	ADMIN_REASON_REQUIRED

```
def execute_settlement_if_allowed(raffle_id):
    gates = evaluate_settlement_gates(raffle_id)
    if not gates.all_passed:
        append_audit_event('SETTLEMENT_BLOCKED', 'raffle', raffle_id, SYSTEM, None, 'BLOCKED',
        gates.to_payload())
        return
    begin_transaction()
    batch = create_settlement_ledger_entries(raffle_id)
    call_psp_split_or_mark_manual(batch)
    append_audit_event('SETTLEMENT_EXECUTED', 'raffle', raffle_id, SYSTEM, 'READY_TO_SETTLE',
    'SETTLED', {'ledger_batch': batch.id})
    commit_transaction()
```

D.6 Security, scopes y admin governance

El control de acceso debe separar scopes de usuario, cliente, admin e internal service. Admin requiere MFA y razón obligatoria para acciones sensibles. El sistema debe registrar intentos rechazados porque también son evidencia relevante para seguridad y auditoría.

Scope	Allowed domains	Controles
user:purchase	orders propios, tickets propios	JWT, rate limit, idempotency
client:raffle	raffles propios, evidence	ownership check, client status
admin:override	sorteos, disputas, settlement	MFA, reason_required, audit
service:worker	reconcile, recovery, outbox	service token, IP allowlist
webhook:psp	payment webhooks	signature, replay protection

D.7 Data retention, privacidad y evidencia

La auditoría fuerte no significa retener todo sin control. Deben existir políticas por clase de dato: audit_events y ledger tienen retención larga; payloads de pago deben minimizarse; evidencia de delivery debe guardarse con hash y ACL; logs operativos deben redaccionar datos sensibles.

Data class	Retención	Protección	Acceso
audit_events	long-term según política legal	append-only + payload_hash	Admin audit
ledger_entries	long-term financiero	immutable + batch integrity	Finance/Admin
payment payloads	minimizada	tokenización / referencias PSP	Finance/Support limitado
delivery evidence	policy-based	object storage ACL + file_hash	Cliente/Admin/Usuario ganador
application logs	30-180 días	redaction + trace_id	SRE/Admin

D.8 Backoffice operacional

El backoffice es la herramienta de gobierno del sistema. No debe ser una simple consola de datos: debe permitir revisión, auditoría, resolución y acciones controladas. Cada acción de backoffice que cambie estado debe exigir permisos, motivo, registro y, cuando corresponda, doble aprobación futura.

Pantalla	Propósito	Acciones críticas	Auditoría
Raffle review	aprobar/observar/cancelar	approve, reject, cancel	RAFFLE_APPROVED/ CANCELLED
Order support	trazar pagos y tickets	reconcile, refund request	ORDER_RECONCILED
Draw audit	ver proof y replay	verify proof, controlled retry	DRAW_PROOF_VIEWED
Dispute console	resolver casos	resolve client/user	DISPUTE_RESOLVED
Finance reconciliation	resolver mismatch PSP-ledger	mark resolved	RECONCILIATION_RESOLVED

D.9 Observability dashboards

Cada dominio debe tener dashboard orientado a decisión. Los dashboards se alimentan de métricas y eventos, no de queries aisladas no versionadas. Una alerta debe tener runbook, severidad, owner y condición de cierre.

Dashboard	Métricas	Audiencia	Acción
Purchase Ops	success rate, PSP lag, duplicate webhook	Ops/SRE	revisar PSP y workers
Draw Integrity	proof failures, duplicate attempts, execution latency	Admin/Tech	bloquear settlement si falla proof
Settlement Finance	aging, payout failures, mismatch	Finance	abrir reconciliation case
Risk	chargebacks, velocity alerts,	Risk/Admin	manual review/KYC futuro

	frozen wallets		
Audit	audit append failures, replay latency	Compliance/Tech	bloquear acciones sensibles si falla audit

D.10 Escalabilidad LATAM y multi-país

La expansión LATAM exige parametrizar moneda, PSP, fee, timezone, impuestos, reglas de evidencia, configuración de risk y settlement. El core debe evitar constantes hardcoded y usar tablas de configuración por país, categoría y proveedor.

Dimensión	MVP Perú	Futuro LATAM
Currency	PEN	multi-currency
PSP	Mercado Pago	PSP por país
Fee	20% fijo + PSP	fee por categoría/país
Timezone	America/Lima	por mercado
Settlement	post-confirmación	escrow/split localizado
Risk	reglas base	modelos por mercado

D.11 Release governance y feature flags

El release debe minimizar riesgo mediante flags, canary, rollback y monitoreo post deploy. Las migraciones de DB deben ser backward-compatible y cualquier cambio de algoritmo Draw debe versionarse explícitamente con algorithm_version para no romper verificabilidad histórica.

Práctica	Propósito	Métrica/Auditoría
feature flags	activar por segmento	FEATURE_FLAG_CHANGED
canary deploy	reducir blast radius	error rate delta
blue/green	rollback rápido	DEPLOYMENT_SWITCHED
safe migrations	evitar downtime	MIGRATION_APPLIED
algorithm versioning	preservar proof histórico	DRAW_ALGORITHM_VERSIONED

D.12 Checklist final de producción

Antes de lanzamiento, el producto debe pasar una checklist de invariantes. Esta lista es una barrera de calidad: si algún punto crítico falla, el MVP no debe salir a producción porque compromete confianza, dinero o trazabilidad.

Checklist	Criterio
Audit append-only	todo proceso crítico genera audit_event
Ledger balanced	debits = credits por batch
Draw reproducible	verify_draw pasa con proof publicado
Settlement gated	no ejecuta con disputa abierta
Webhooks idempotentes	no duplican tickets ni ledger
Observability	dashboards y alertas activos
Recovery jobs	órdenes pending se reconcilian
Admin governance	overrides requieren motivo y audit

ANEXO E - Secuencia operacional por trace_id

El trace_id es la columna vertebral de la observabilidad y auditoría. ALAZAR debe asignar o propagar un trace_id desde el primer request del Usuario o Cliente hasta todos los eventos internos, outbox, ledger, PSP correlation y logs. Esto permite reconstruir una compra completa sin depender de búsqueda manual. El objetivo no es solo debugging; es defensa operacional y cumplimiento.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

ANEXO F - Modelo de datos versionado y migraciones

El PRD debe asumir que el modelo de datos evolucionará. Por ello se definen estrategias para versionar eventos, versionar draw algorithms, migrar tablas con backward compatibility y mantener compatibilidad de APIs. Las migraciones no deben destruir trazabilidad histórica ni invalidar pruebas de draws ejecutados.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

ANEXO G - Política de consistencia por dominio

No todos los dominios requieren el mismo tipo de consistencia. Purchase, ticket issuance, ledger y draw requieren consistencia fuerte en escritura. Dashboards, notificaciones y métricas pueden usar consistencia eventual. Este criterio evita sobrearquitecturar lo no crítico y protege lo que sí impacta dinero o fairness.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

ANEXO H - Modelo de notificaciones y comunicación

Notificaciones no son parte decorativa: informan estados críticos a Usuario, Cliente y Admin. Cada notificación debe derivarse de eventos de dominio, no de cronjobs ad hoc. La notificación debe tener canal, template, evento origen, actor destino y estado de entrega.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

ANEXO I - Modelo de administración de evidencias

Las evidencias de delivery, disputa y auditoría deben almacenarse con hash, metadata, owner y permisos. Se requiere control de acceso porque una evidencia puede contener información sensible. El sistema no debe depender del nombre del archivo, sino del file_hash y del vínculo a delivery_case o dispute_case.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

ANEXO J - Matriz de criterios de aceptación por capítulo

Cada capítulo del PRD debe tener criterios de aceptación funcional, técnico y operativo. Esto permite que QA y desarrollo validen cumplimiento sin reinterpretar la intención del documento. La matriz también sirve para priorizar MVP frente a funcionalidades post-MVP.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

ANEXO K - Runbooks de incidentes críticos

Los runbooks convierten incidentes repetibles en procedimientos controlados. Un incidente de PSP, de draw proof, de audit append o de settlement mismatch debe tener pasos, owners, métricas y condición de cierre. Sin runbooks, la plataforma dependería de criterio improvisado durante incidentes.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

ANEXO L - Checklist CTO para Go-Live

El Go-Live debe aprobarse contra invariantes de arquitectura, seguridad, auditoría, ledger, draw, settlement, infraestructura y observabilidad. Esta checklist debe firmarse internamente antes del MVP público.

La siguiente matriz convierte el concepto en requisitos operativos. Cada fila debe implementarse como historia técnica o criterio de QA. Si una fila implica dinero, tickets, ganador, evidencia, disputa o override, debe producir audit_event y log estructurado con trace_id.

Requisito	Implementación	Validación	Auditoría
trace_id obligatorio	middleware genera/propaga X-Correlation-ID	test de flujo end-to-end	TRACE_PROPAGATED
event_version	schema version en eventos críticos	contract tests	EVENT_VERSION_RECORDED
strong consistency	transacción DB en dominios críticos	integration tests	DOMAIN_TX_COMMITTED
eventual consistency	outbox + workers para secundarios	worker tests	OUTBOX_PUBLISHED
access control	RBAC + scopes + ownership	security tests	AUTHZ_DECISION
immutable evidence	file_hash + metadata + ACL	evidence retrieval test	EVIDENCE_STORED
incident response	runbook + owner + cierre	game day	INCIDENT_RECORDED

```
def enterprise_guardrail(action, entity, actor):
    trace_id = current_trace_id()
    assert trace_id is not None
    assert authorize(actor, action, entity)
    if action.impact_money_or_fairness:
        assert audit_service.available()
    result = execute_action(action, entity)
    append_audit_event(action.audit_event, entity.type, entity.id, actor, action.prev_state,
action.new_state, result.payload)
    emit_outbox_event(action.domain_event, result.payload)
    return result
```

La implementación debe evitar soluciones ad hoc. La misma regla de guardrail debe aplicar para endpoints públicos, internos, workers y backoffice. Esta uniformidad reduce deuda técnica y hace que el sistema pueda escalar sin perder control.

PARTE XI - HARDENING ENTERPRISE v11: elevación a fintech production-grade

Esta parte se agrega de forma acumulativa sobre el PRD v11. No reemplaza el contenido anterior: lo extiende para cerrar brechas enterprise en APIs, ledger, seguridad, infraestructura, observabilidad, QA y resiliencia. El objetivo es llevar el documento de un PRD fintech enterprise sólido a un blueprint productivo más cercano a un handbook de arquitectura interna, con controles explícitos para auditoría, trazabilidad, operación y escalabilidad.

Área	Estado v11	Hardening v11	Resultado esperado
APIs	Enterprise base	OpenAPI-style, scopes, signatures, rate limits, pagination, error envelope	Contratos implementables
Ledger	Double-entry sólido	Reservas, snapshots, close, multicurrency-ready, chargebacks	Bank-grade readiness
Security/Risk	Capa transversal	Threat model, anti-bot, ATO, device, payout risk	Control antifraude operacional
Infra/Obs	Operable	K8s topology, DR, DLQ, SLOs, runbooks, secrets	Production-grade operations
QA/Resilience	Criterios MVP	Chaos, replay, contract, fairness fuzzing, failure simulation	Release gates reales

XI.1 APIs Enterprise - OpenAPI-style, contratos y gobernanza

La capa API debe funcionar como contrato formal entre frontend, backoffice, workers internos, PSP y consumidores externos. Para operar como plataforma fintech, cada endpoint que modifique estado debe tener autenticación, authorization scopes, Idempotency-Key, trace_id, error envelope uniforme, límites de tasa y efecto de auditoría. Las APIs no son solamente rutas HTTP: son puntos de control operacional y de evidencia.

OpenAPI Enterprise Gateway - Auth, Idempotency, Audit, Rate Limit

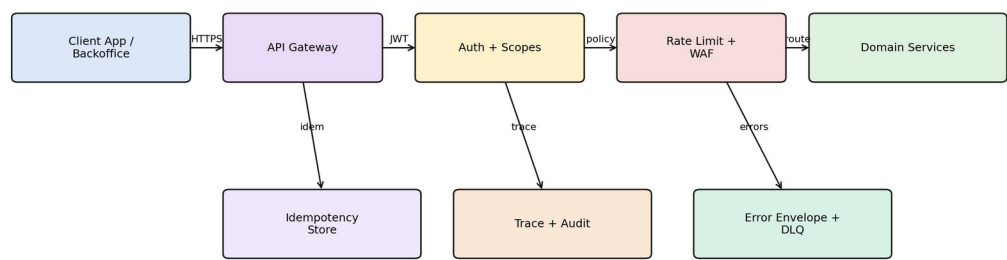


Figura XI.1. Gateway API enterprise con auth, idempotencia, auditoría y rate limiting.

Header	Requerido en	Propósito	Auditoría
Authorization: Bearer	todas las APIs privadas	identidad y scopes	actor_id, actor_type
Idempotency-Key	POST/PUT con efecto	evitar duplicidad	IDEMPOTENCY_REUSED o CREATED
X-Correlation-Id	todas	tracing distribuido	trace_id
X-Client-Version	frontend/app	diagnóstico de releases	client_version
X-Signature	webhooks	firma payload	WEBHOOK_SIGNATURE_VERIFIED

Dominio	Endpoint	Scope	Rate limit sugerido	Audit event
Purchase	POST /v1/orders/purchase	orders:write	10/min/user	ORDER_INTENT_CREATED
Draw	POST /v1/draw/execute	draw:execute	3/min/admin	DRAW_REQUESTED
Delivery	POST /v1/delivery/{id}/evidence	delivery:write	20/min/client	DELIVERY_EVIDENCE_UPLOADED
Settlement	POST /v1/settlements/{id}/execute	settlement:execute	5/min/system	SETTLEMENT_TRIGGERED
Audit	GET /v1/admin/audit	audit:read	30/min/admin	AUDIT_VIEWED
Webhook	POST /v1/payments/webhooks/mercadopago	webhook:ingest	provider controlled	WEBHOOK_RECEIVED

```
Error envelope estándar:
{
  "error_code": "ERR_PURCHASE_003",
  "message": "Not enough tickets available",
  "trace_id": "trc_abc123",
  "retryable": false,
  "recoverable_by": "USER_ACTION",
  "details": { "raffle_id": "rf_123", "available": 2 }
}
```

Los errores deben ser estables y versionados porque frontend, backoffice y soporte dependerán de ellos. Cada error crítico debe mapearse a una acción esperada: retry, soporte, refresh, reautenticación, revisión administrativa o bloqueo definitivo.

OpenAPI/Webhook sequence - signed, idempotent, replay-safe

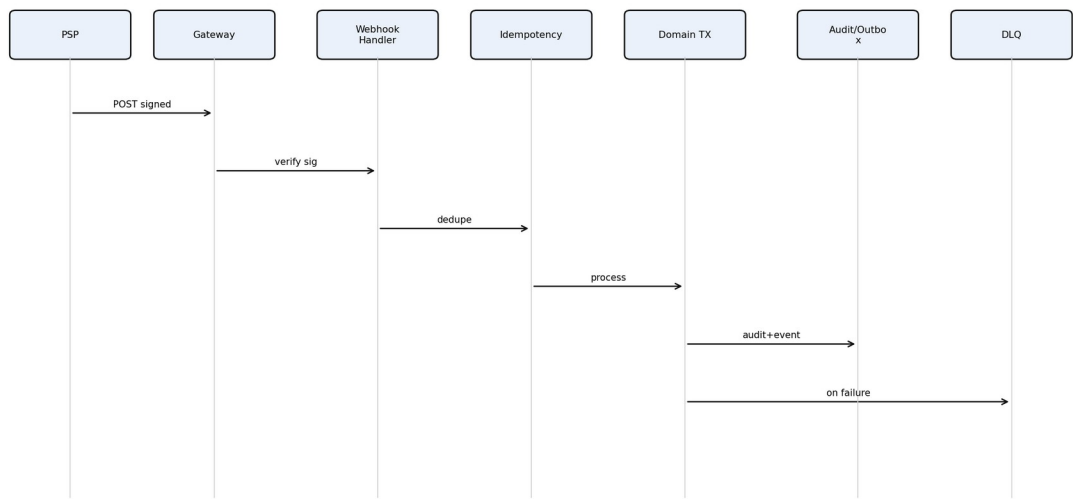


Figura XI.2. Secuencia de webhook firmado, idempotente y replay-safe.

```
def verify_webhook(payload, headers):
    signature = headers.get("X-Signature")
    timestamp = headers.get("X-Timestamp")
    assert within_replay_window(timestamp, max_age_seconds=300)
    expected = hmac_sha256(secret, canonical_json(payload) + timestamp)
    if not constant_time_equals(signature, expected):
        append_audit("WEBHOOK_SIGNATURE_FAILED", entity_id=payload.get("id"))
        raise UnauthorizedWebhook()
    return True

def ingest_webhook(payload, headers):
    verify_webhook(payload, headers)
    event_id = payload["id"]
```

```
if webhook_processed(event_id):
    append_audit("WEBHOOK_DUPLICATE_IGNORED", entity_id=event_id)
    return {"status": "ignored"}
enqueue_webhook(event_id, payload)
append_audit("WEBHOOK_ACCEPTED", entity_id=event_id)
```

XI.2 Ledger bank-grade y cierre contable

El ledger debe pasar de un registro contable suficiente para MVP a un modelo bank-grade future-ready. Esto implica separar cash clearing, liabilities, reserves, platform revenue, client payable, refund reserve, chargeback loss y accounting snapshots. La meta no es convertir ALAZAR en banco, sino evitar rediseñar la contabilidad cuando escale a múltiples PSPs, países, monedas o modelos de escrow/split real.

Bank-grade Ledger - accounts, reserves, settlement, close

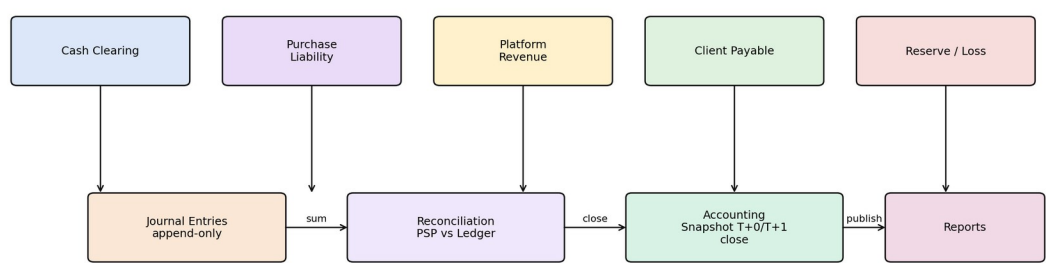


Figura XI.3. Modelo ledger bank-grade con reservas, snapshot y cierre contable.

Cuenta	Tipo	Uso	Ejemplo	
Cash Clearing	Activo transitorio	fondos confirmados por PSP pendientes de asignar	payment.approved	
Purchase Liability	Pasivo	obligación por tickets y premios hasta cierre	compra confirmada	
Platform Revenue	Ingreso	fee ALAZAR reconocido según política	fee 20%	
Client Payable	Pasivo	monto neto pendiente de pago al cliente	post entrega	
Refund Reserve	Reserva	monto retenido para reversiones	disputa usuario	
Chargeback Loss	Gasto/pérdida	absorción de contracargos	chargeback PSP	
FX Clearing	Transitoria futura	diferencias de moneda	expansión LATAM	
Flujo	Debit	Credit	Condición	Audit event
Purchase approved	Cash Clearing	Purchase Liability	PSP approved	PURCHASE_COMPLETED
Platform fee recognition	Purchase Liability	Platform Revenue	policy satisfied	FEE_RECOGNIZED
Settlement client	Purchase Liability	Client Payable	delivery confirmed	SETTLEMENT_TRIGGERED
Payout executed	Client Payable	Cash Clearing	PSP payout ok	PAYOUT_EXECUTED
Refund partial	Refund Reserve	Cash Clearing/User Balance	dispute resolved user	REFUND_EXECUTED
Chargeback	Chargeback Loss	Cash Clearing	PSP chargeback	CHARGEBACK_RECORDED

```
def post_journal(event):
    entries = journal_template(event.type, event.amount, event.currency)
    assert sum(e.debit for e in entries) == sum(e.credit for e in entries)
```

```

for entry in entries:
    insert_ledger_entry(
        account=entry.account,
        debit=entry.debit,
        credit=entry.credit,
        currency=event.currency,
        reference_id=event.reference_id,
        trace_id=event.trace_id
    )
    append_audit("LEDGER_JOURNAL_POSTED", entity_id=event.reference_id, metadata={"entries":
len(entries)})

def accounting_close(day):
    freeze_ledger_window(day)
    reconcile_psp_vs_ledger(day)
    create_accounting_snapshot(day)
    publish_finance_report(day)
    append_audit("ACCOUNTING_CLOSE_COMPLETED", entity_id=str(day))

```

El cierre contable debe generar snapshots inmutables por periodo. Si una corrección llega después del cierre, no se edita el pasado: se registra una entrada compensatoria en el periodo actual. Esto preserva trazabilidad y evita manipulación histórica.

XI.3 Security, anti-fraud y threat modeling

La seguridad debe cubrir amenazas técnicas y abuso de negocio. En ALAZAR no basta con proteger login: también se deben proteger compra masiva, manipulación de LIVE, abuso de disputas, multi-accounting, bots, chargebacks y riesgo de payout. El Risk Engine debe influir sobre Purchase, Draw, Delivery y Settlement, generando evidencia auditada de cada decisión.

Security Threat Model - entrypoints, controls, evidence

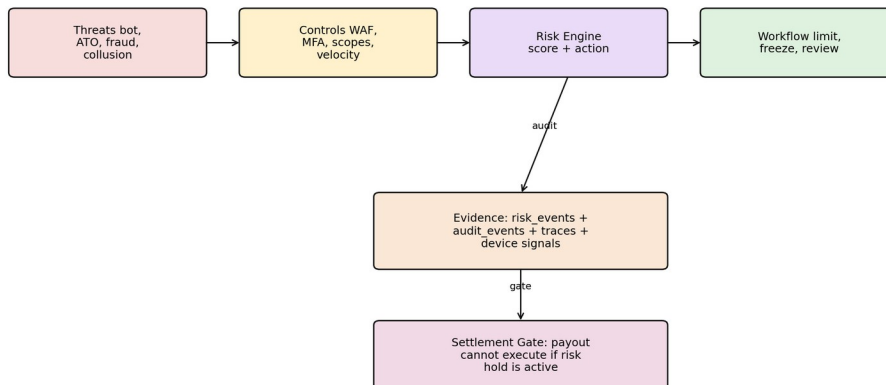


Figura XI.4. Threat model con controles, Risk Engine y bloqueo de settlement.

Amenaza	Vector	Control preventivo	Control detectivo	Respuesta
Bot purchase	alta frecuencia checkout	rate limit, captcha adaptativo	velocity anomaly	throttle/freeze
Account takeover	credential stuffing	MFA, device binding	login anomaly	session revoke
Multi-accounting	mismo device/IP	device fingerprint	cluster detection	soft block/KYC
LIVE abuse	cliente ejecuta sin permiso	admin authorization	LIVE_TRIGGER audit	reject/alert

Dispute abuse	usuario disputa todo	dispute history score	pattern detection	manual review
Payout fraud	cliente riesgoso	client risk score	chargeback/dispute ratio	settlement hold

```
def risk_decision(actor, action, context):
    signals = collect_signals(actor, context)
    score = (signals.velocity*0.25 + signals.device_risk*0.20 +
             signals.chargeback_risk*0.25 + signals.dispute_risk*0.20 +
             signals.geo_anomaly*0.10)
    if score >= 85:
        decision = "BLOCK"
    elif score >= 70:
        decision = "HOLD_FOR_REVIEW"
    elif score >= 50:
        decision = "LIMIT"
    else:
        decision = "ALLOW"
    append_audit("RISK_DECISION_APPLIED", entity_id=context.entity_id, metadata={
        "action": action, "score": score, "decision": decision, "signals_hash": hash(signals)
    })
    return decision
```

Todo bloqueo debe ser reversible y justificable. La plataforma debe registrar señales, decisión, actor, entidad, efecto operativo y efecto financiero. Si Risk congela una wallet o settlement, Settlement debe leer ese estado antes de liberar cualquier payout.

XI.4 Infraestructura hyperscale, resiliencia operacional y DR

La infraestructura debe soportar crecimiento sin perder control operativo. Para MVP puede operar en una arquitectura simple, pero el diseño debe prever Kubernetes, despliegues blue/green, secretos centralizados, outbox/inbox, DLQ, backups con PITR, observabilidad distribuida y separación de ambientes. La arquitectura no debe depender de intervención manual para recuperarse de fallos comunes.

Hyperscale Infrastructure - resilient event-driven topology

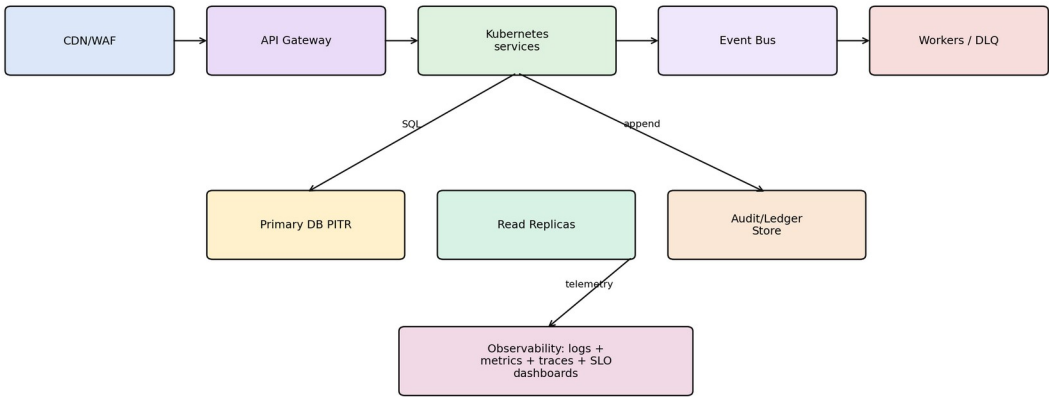


Figura XI.5. Topología hyperscale-ready con gateway, K8s, event bus, DLQ, DB y observabilidad.

Componente	Política enterprise	Riesgo mitigado
API Gateway	WAF, rate limit, JWT validation	abuso y tráfico malicioso
Kubernetes	autoscaling, rolling/blue-green deploy	caídas por release
Outbox/Inbox	publicación confiable + idempotencia consumer	event loss/duplicates
DLQ	quarantine luego de N retries	fallos silenciosos

DB	PITR, backups diarios, read replicas	pérdida de datos
Secrets	Vault/KMS, rotation	exposición credenciales
Observability	logs, metrics, traces, SLOs	MTTR alto

```

circuit_breaker(service):
    if error_rate(service) > threshold or latency_p95(service) > sla:
        open_circuit(service)
        route_to_fallback_or_queue(service)
        append_audit("CIRCUIT_BREAKER_OPENED", entity_id=service)

```

```

retry_policy = {
    "max_attempts": 5,
    "backoff": "exponential",
    "jitter": True,
    "send_to_dlq_after": 5
}

```

SLO	Objetivo	Alerta
purchase_success_rate	>= 99% de intents válidos	< 97% en 15 min
webhook_processing_lag_p95	< 60s	> 120s
draw_execution_p95	< 5s	> 10s
audit_append_success	>= 99.99%	cualquier caída sostenida
settlement_aging	< SLA configurado	casos vencidos
reconciliation_mismatch_rate	< 0.1%	> umbral

XI.5 QA enterprise, resilience validation and release gates

QA debe validar invariantes de negocio, no solo pantallas. El sistema no puede salir a producción si permite tickets sin ledger, draw sin pool freeze, settlement con disputa abierta, webhook duplicado con doble efecto o auditoría incompleta. La calidad enterprise requiere pruebas unitarias, contract testing, replay testing, load/soak, chaos engineering y validación de resiliencia.

QA and Resilience - validation matrix and gates

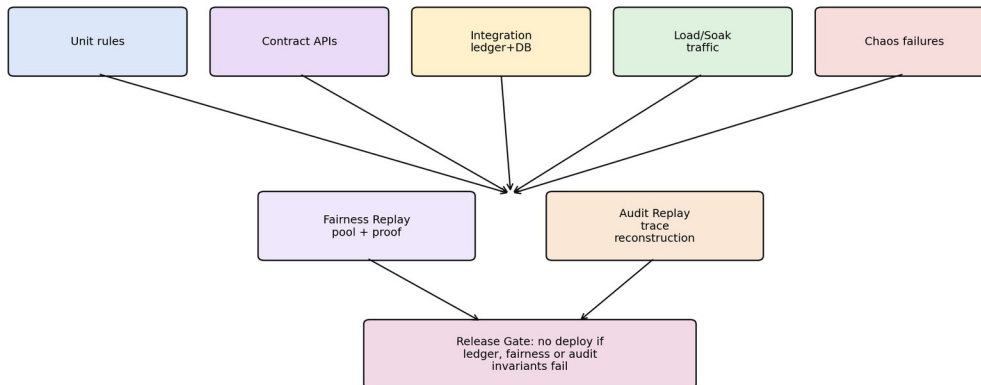


Figura XI.6. QA y resiliencia: release gate basado en invariantes de auditoría, ledger y fairness.

Tipo de prueba	Qué valida	Ejemplo de prueba	Release gate
Unit	reglas puras	pricing, RBAC, state transitions	100% reglas críticas
Contract	APIs	schemas, error envelope, idempotency	sin breaking changes
Integration	DB + servicios	purchase -> ledger -> tickets	invariantes OK
Replay	auditoría	trace id reconstruye timeline	100% procesos críticos
Fairness fuzzing	Draw Engine	pool aleatorio reproducible	proof siempre verifica
Load/Soak	concurrency	últimos tickets simultáneos	sin oversell

Chaos	fallos PSP/DB/queue	recovery + DLQ	sin corrupción
<pre> def invariant_purchase(order_id): order = get_order(order_id) tickets = get_tickets(order_id) ledger = get_ledger_entries(order_id) audit = get_audit_events(order_id) assert order.status in ["PAID", "COMPLETED"] assert len(tickets) == order.qty assert sum_debits(ledger) == sum_credits(ledger) assert has_event(audit, "PURCHASE_COMPLETED") def invariant_draw(draw_id): proof = get_draw_proof(draw_id) assert verify_draw(draw_id) == True assert has_event(get_audit_events(draw_id), "DRAW_EXECUTED") def release_gate(build_id): run_contract_tests() run_replay_tests() run_fairness_fuzzing() run_ledger_consistency_suite() if any_failure(): block_deploy(build_id) </pre>			

La estrategia de despliegue debe usar feature flags para activar gradualmente raffle types, LIVE execution, settlement automático y nuevas integraciones PSP. Cada release debe poder apagarse por dominio sin tumbar toda la plataforma.

XI.6 Matriz final de madurez enterprise

La siguiente matriz resume cómo las capas v11 elevan el documento a un estándar más cercano a handbook enterprise. No reemplaza los capítulos previos, sino que demuestra la cobertura transversal alcanzada.

Área	Cobertura v11	Nivel esperado
Arquitectura funcional	C4 + event-driven + bounded contexts + workflows end-to-end	9.6+
Purchase Engine	idempotencia, PSP, ledger, recovery, reconciliation, audit	9.6+
Draw Engine	crypto-grade, external entropy, proof, replay, LIVE controlled	9.6+
Auditoría	append-only, trace_id, hash, lineage, forensic reconstruction	9.7+
APIs	OpenAPI-style, scopes, signatures, pagination, rate limit	9.5+
Ledger	bank-grade accounts, reserves, close, snapshots	9.5+
Delivery/Settlement	evidence, dispute, payout gates, escrow-ready	9.5+
Observabilidad	SLO, dashboards, tracing, DLQ, recovery metrics	9.4+
Infraestructura	K8s-ready, DR, backups, secrets, circuit breakers	9.3+
Security/QA	threat model, risk scoring, chaos, replay, fairness fuzzing	9.4+

Con esta expansión, el PRD deja de ser solamente una especificación de producto y se convierte en un blueprint técnico-operacional para construir ALAZAR con estándares de plataforma fintech moderna. El siguiente refinamiento posible ya no sería reescritura del PRD, sino anexos especializados: OpenAPI YAML completo, DDL SQL de producción, runbooks SRE y threat model formal STRIDE/abuse-case.